

UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE CIENCIAS FÍSICAS

Máster Universitario en Nuevas Tecnologías Electrónicas y Fotónicas



TRABAJO DE FIN DE MÁSTER

Desarrollo de un supercomputador de bajo coste y bajo consumo para aplicaciones científicas

Alejandro Otero Vázquez

Tutores: Guillermo Botella Juan y Sandra Catalán Pallarés

Departamento de Arquitectura de Computadores y Automática.
Facultad de Ciencias Físicas.

Convocatoria Septiembre de 2023-24

Resumen:

En este Trabajo Fin de Máster se ha construido un prototipo de un computador de altas prestaciones escalable y modular de bajo coste mediante varias placas Raspberry Pi 4. La máquina que se presenta pretende demostrar que con un presupuesto reducido se pueden llevar a cabo proyectos de supercomputación.

Para las pruebas y la evaluación del rendimiento, se utilizaron los benchmarks ofrecidos por el distribuidor GROMACS. Mediante estos benchmarks se puso a prueba al clúster variando la cantidad de nodos de cómputo (1, 2 y 3) y las prestaciones hardware intra-nodo. Midiendo el consumo energético frente a la capacidad de cómputo del dispositivo durante las pruebas de rendimiento se pudo caracterizar el desempeño del sistema. Esto permitió poner de manifiesto el aumento en las prestaciones y eficiencia del clúster mediante la aplicación de software especializado en el paralelismo y la concurrencia dentro de un ámbito científico.

Con los resultados obtenidos, se analizó el rendimiento del clúster en comparación con otros sistemas más tradicionales, destacando la relación favorable prestaciones/coste del uso de placas Raspberry Pi en la construcción de supercomputadores. Se espera que este Trabajo Fin de Máster no solo sirva como una guía práctica para la creación de clústers de bajo coste y bajo consumo, sino que también inspire futuras investigaciones y proyectos en el campo de la computación de altas prestaciones con recursos limitados.

Abstract:

In this Master's thesis, a scalable and modular high-performance computer has been built at a low cost using several Raspberry Pi 4 boards. The supercomputer presented here aims to demonstrate that, with a limited budget, it is possible to undertake projects of high-performance computing.

Once the cluster is constructed, it will be evaluated using various metrics such as computing capacity, energy consumption, and more. For this purpose, a proof of concept will be carried out with a comprehensive evaluation of a scientific application, highlighting the extraction of parallelism in scientific fields.

Furthermore, the cluster's performance will be analyzed in comparison to more traditional systems, emphasizing the advantages and limitations of using Raspberry Pi boards in the construction of supercomputers. This work is expected not only to serve as a practical guide for creating low-cost and low-consumption clusters but also to inspire future research and projects in the field of high-performance computing with limited resources.

Palabras clave:

Supercomputador, Computación de altas prestaciones, Raspberry Pi, Clúster, Paralelismo, Concurrencia, MPI, OpenMP, Nodo, DHCP, Capacidad de cómputo, Consumo energético.

Keywords:

Supercomputer, High-performance computing, Raspberry Pi, Cluster, Parallelization, Concurrency, MPI, OpenMP, Node, DHCP, Computing capacity, Energy consumption.

Índice

1. Introducción.	1
1.1. Motivaciones.	2
1.2. Objetivos.	3
1.3. Plan de trabajo	5
2. Estado del arte.	5
2.1. Uso de Raspberry Pi en HPC.	6
2.2. Concurrencia y paralelismo.	7
2.3. Modelos de programación paralela.	9
2.4. Intra-nodo: OpenMP.	10
2.5. Inter-nodo: MPI.	11
3. Montaje del clúster.	13
3.1. Coste económico.	15
4. Configuración del clúster.	16
4.1. Creación de una imagen ejecutable de la tarjeta SD.	17
4.2. Puesta en marcha de cada nodo del clúster.	18
4.3. Sistema de ficheros compartidos.	21
4.4. Modelos de programación paralela.	22
5. Evaluación del rendimiento del clúster.	24
5.1. Benchmarks empleados para la evaluación del rendimiento del clúster.	24
5.2. Métricas empleadas para la evaluación.	25
5.3. Experimentos realizados para evaluación del consumo energético frente al tiempo de ejecución para distintas configuraciones del clúster.	28
5.3.1. Experimento 1. Influencia de la frecuencia de trabajo de la CPU sobre el consumo energético del chip y su capacidad de cómputo.	28
5.3.2. Experimento 2. Influencia de la frecuencia de trabajo de la CPU sobre el consumo energético del chip y su capacidad de cómputo para múltiples núcleos.	31
5.3.3. Experimento 3. Influencia del número de núcleos de la CPU activos sobre el consumo energético del chip y su capacidad de cómputo.	33
5.3.4. Experimento 4. Influencia del número de procesos MPI en los que se diversifique la carga sobre el consumo energético del chip y su capacidad de cómputo. Experimento realizado para un único nodo.	35

5.3.5. Experimento 5. Estudio de la capacidad de cómputo y consumo energético del clúster al utilizar distintas configuraciones cuando todos los nodos de cómputo están activos.	38
5.3.6. Experimento 6. Optimización de la capacidad de cómputo con el consumo energético del clúster para distintas configuraciones cuando todos los nodos de cómputo están activos.	40
5.3.7. Curva de Pareto	41
6. Conclusiones y trabajo futuro.	45

Índice de figuras

1.	Objetivos específicos que se pretenden abordar dentro de cada etapa del plan de trabajo. Fuente: elaboración propia.	4
2.	Esquema gráfico de los efectos sobre la cantidad de recursos de la CPU dedicados a la ejecución tras el uso de OpenMP. Fuente: elaboración propia.	11
3.	Esquema de comunicaciones y funcionalidades de MPI en el clúster desarrollado en este TFM de cuatro nodos Raspberry Pi, con cuatro cores cada uno. Fuente: elaboración propia.	12
4.	Ejemplos de topologías en redes de comunicación. Fuente: https://247tecnologia.com/topologia-de-red-tipos-caracteristicas/	14
5.	Fotografía del clúster de Raspberry Pi construido.	15
6.	Diagrama de flujo que describe los procesos necesarios para la configuración del clúster. Fuente: elaboración propia.	17
7.	Esquema de la estructura de un clúster con cuatro nodos de cómputo conectados en topología de estrella a través de un switch. Fuente: Elaboración propia.	22
8.	Diagrama de flujo de los experimentos de caracterización del clúster.	29
9.	Evolución de la temperatura de la CPU con el tiempo para distintas frecuencias de trabajo del procesador. Experimento realizado para el <i>front-end</i> sometándolo a una carga computacional controlada 5.1. Los tiempos de ejecución para cada configuración vienen recogidos siguiendo el código de colores de la leyenda.	30
10.	Evolución de la temperatura de la CPU con el tiempo para distintas frecuencias de trabajo del procesador cuando se somete a una carga computacional controlada 5.1. Experimento realizado para el <i>nodo1</i> . Los tiempos de ejecución para cada configuración vienen recogidos siguiendo el código de colores de la leyenda.	30
11.	Evolución de la temperatura con el tiempo de la CPU de tres chips, emulando ser núcleos con control independiente dentro de un mismo procesador. Se estudia la influencia de la frecuencia de trabajo de cada núcleo sobre los tiempos de ejecución cuando se somete al procesador a una carga computacional controlada 5.1. La frecuencia de trabajo de cada “núcleo” se recoge en la leyenda siguiendo el código de colores de cada curva.	32
12.	Evolución de la temperatura con el tiempo de la CPU de un nodo de cómputo cuando su procesador está sometido a una carga computacional controlada 5.1. Simulación llevada a cabo variando el número de threads OpenMP en los que se paraleliza la simulación. El número de núcleos del procesador (o threads) implicados se recoge en la leyenda siguiendo su código de colores.	34

13.	Evolución de la temperatura con el tiempo de la CPU de un nodo de cómputo cuando se somete al procesador a una carga computacional controlada 5.1. Simulación llevada a cabo variando el número de procesos en los que se paraleliza la simulación. El número de procesos implicados se recoge en la leyenda siguiendo el código de colores de las curvas.	36
14.	Evolución de la temperatura con el tiempo de la CPU de los tres nodos de cómputo cuando se somete al procesador a una carga computacional controlada 5.1. Se han llevado a cabo simulaciones variando el número de procesos MPI manteniendo en todas las simulaciones el número de hilos totales constante e igual al número de núcleos de los que disponibles en el clúster (12 cores).	39
15.	Tiempo de ejecución y consumo energético del espacio de configuraciones del clúster cuando se somete al procesador a una carga computacional controlada 5.1. La curva de Pareto recorre las configuraciones que minimizan el consumo energético para cada tiempo de ejecución.	42

Índice de tablas

1.	Especificaciones de la placa Raspberry Pi 4 Model B empleada en el clúster.	14
2.	Lista de componentes con sus cantidades y precios.	16
3.	IP estática para cada nodo.	19
4.	Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en un nodo de cómputo durante el experimento 1. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 10.	31
5.	Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en los tres nodos de cómputo durante el experimento 2. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 11.	33
6.	Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en los tres nodos de cómputo durante el experimento 3. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 12.	35
7.	Comparación de los tiempos de ejecución y eficiencias relativas para los procesadores emulados en el experimento 2 (núcleos pertenecientes a distintos procesadores) con los núcleos de un solo procesador empleados en el experimento 3 y con el experimento 4, en el que se emplean procesos en lugar de hilos.	36
8.	Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en los tres nodos de cómputo durante el experimento 4. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 13.	37
9.	Comparación de los tiempos de ejecución y eficiencias relativas para las distintas configuraciones probadas durante el experimento 5. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 14.	38
10.	Información de todas las configuraciones empleadas durante el experimento 6. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 15.	44

Lista de Acrónimos.

- **HPC:** High Performance Computing
- **CPU:** Central Processing Unit
- **GPU:** Graphics Processing Unit
- **MPI:** Message Passing Interface
- **OpenMP:** Open Multi-Processing
- **LINPACK:** Linear Algebra Package
- **FLOPS:** Floating Point Operations Per Second
- **NUMA:** Non-Uniform Memory Access
- **SMP:** Symmetric Multi-Processing
- **SSH:** Secure Shell
- **SLURM:** Simple Linux Utility for Resource Management
- **OS:** Operating System
- **IP:** Internet Protocol
- **NFS:** Network File System
- **ARM:** Advanced RISC Machines
- **DHCP:** Dynamic Host Configuration Protocol
- **API:** Application Programming Interface
- **SDRAM:** Synchronous Dynamic Random-Access Memory
- **SDHC:** Secure Digital High Capacity
- **MPICH:** Message Passing Interface Chameleon

1. Introducción.

El creciente aumento sobre el número y complejidad de las aplicaciones computacionales, que se ha observado en los últimos años, ha derivado en el desarrollo de nuevas arquitecturas de computadores basadas en el uso de máquinas con múltiples unidades de procesamiento que cooperen entre sí. Estas arquitecturas se conocen en el mundo de la computación de altas prestaciones como clústers. La cooperación entre las unidades de procesamiento que conforman un clúster ha derivado en la exploración de nuevas fronteras en la potencia de procesamiento que se puede llegar a desarrollar en la resolución de tareas complejas que demandan un uso de un gran volumen de datos.

Este Trabajo Fin de Máster recoge la implementación de un clúster funcional aplicable a computación de altas prestaciones (HPC) utilizando tarjetas Raspberry Pi 4, modelo B. Los clústers formados por tarjetas Raspberry Pi han demostrado ser una solución viable y económica para llevar a cabo tareas que involucran protocolos específicos dedicados a la computación de altas prestaciones. El montaje de un clúster formado por estas placas no requiere grandes inversiones para su implementación.

Como ya se ha dicho, un clúster generalmente está compuesto por un grupo de computadores independientes (llamados nodos) que trabajan conjuntamente para resolver un problema. Cada uno de esos nodos (computadores) puede disponer de uno o varios procesadores. A su vez, cada procesador dispone de varios cores o núcleos físicos. En caso de emplear placas Raspberry Pi 4 como nodos de cómputo, cada nodo tendrá un solo procesador con cuatro cores. Los nodos, por lo general, presentan las mismas prestaciones a nivel de hardware, aunque existen clústers heterogéneos que emplean nodos con distintas especificaciones. La finalidad de este trabajo será construir un clúster dedicado a HPC que pueda emplear múltiples nodos de cómputo que trabajen conjuntamente en la resolución de un problema concreto (o benchmark).

La clave para construir un clúster capaz de optimizar los recursos hardware de los que dispone es la optimización de los protocolos de colaboración entre las unidades de procesamiento de todos sus nodos. Para que los nodos de cómputo puedan trabajar conjuntamente es necesario dotar al clúster de una red de comunicación que interconecte las distintas partes que lo conforman. La red de comunicación es la base para que se pueda acceder a más recursos dentro del clúster y, con la configuración adecuada, se puedan llegar a aprovechar los recursos de varios o incluso todos los nodos en aplicaciones diseñadas para ello. Sin una red de comunicación el rendimiento de estas máquinas estaría limitado a la paralelización que se pudiera llevar a cabo únicamente dentro de un único nodo.

El concepto de paralelización surge de la necesidad de dividir un problema en tareas que puedan afrontarse de forma independiente por las distintas unidades de procesamiento. No obstante, la paralelización aparece también intra-nodo, entre los distintos cores que conforman cada procesador. Estos cores comparten espacios de memoria. El hecho de tener memoria compartida facilita la comunicación entre los cores porque pueden acceder a la misma información directamente sin necesidad de comunicarse mediante un mecanismo de paso de mensajes para poder distribuirla. En este punto, se podrían definir dos niveles o tipos de paralelismo: intra-nodo con memoria compartida e inter-nodos con lo que se conoce como memoria distribuida.

1.1. Motivaciones.

Una de las principales motivaciones detrás de la construcción de un supercomputador de bajo coste utilizando placas Raspberry Pi 4 es la accesibilidad que presenta gracias a su coste reducido. Tradicionalmente, la computación de altas prestaciones ha estado reservada para instituciones con recursos financieros significativos debido al alto coste del hardware especializado que requiere. En este Trabajo Fin de Máster se ha buscado proponer una alternativa a la barrera económica que supone acceder a la HPC sin contar con grandes recursos.

Este proyecto presenta, además, una fuerte motivación educativa y de formación. La arquitectura de un supercomputador basado en Raspberry Pi ofrece una oportunidad para animar a que los estudiantes y novatos en el campo de la HPC comprendan los conceptos fundamentales de paralelismo, distribución de tareas y configuración de clústers. Este enfoque práctico facilita el aprendizaje de la teoría mediante la implementación y experimentación directa.

En este trabajo se presenta un clúster de cuatro placas, pero gracias a la escalabilidad y flexibilidad que ofrecen las placas Raspberry Pi, la adición o remoción de nodos se puede llevar a cabo con relativa facilidad. Esta característica es esencial para un entorno de investigación y desarrollo donde las necesidades de computación pueden cambiar rápidamente. Se puede comenzar con un número pequeño de nodos y expandir el clúster según sea necesario, lo que proporciona un enfoque eficiente y adaptable.

Este clúster busca ser una alternativa de bajo consumo para aplicaciones científicas que no requieran una carga computacional excesivamente elevada. Las Raspberry Pi 4 son conocidas por su bajo consumo energético, que varía entre 2,7W y 7,6W en función de la carga de trabajo y del uso de periféricos adicionales durante su uso. En comparación, los servidores tradicionales que suelen consumir entre 200W y 500W en reposo, y pueden superar los 1.000W bajo cargas intensivas. Para medir el rendimiento energético de las placas durante la caracterización del clúster, se

ha empleado la eficiencia energética como una métrica de evaluación. Evaluar el rendimiento y consumo energético de un clúster basado en estas placas permite explorar soluciones más sostenibles y ecológicas para HPC, un área de creciente importancia dada la preocupación global por el desarrollo sostenible. Comparado con servidores tradicionales, un clúster de Raspberry Pi 4 puede ofrecer mejores competencias en cuanto a su eficiencia energética, a pesar de que estos clústers generalmente no son capaces de alcanzar la capacidad de procesamiento de los supercomputadores convencionales. Esto hace de estas placas una opción atractiva para tareas de computación distribuida que no requieran alta potencia de cálculo.

El proyecto también pretende servir como una prueba de concepto que demuestre las capacidades y limitaciones de un clúster basado en Raspberry Pi. Debe tenerse en cuenta que estas placas no fueron desarrolladas para utilizarse en este ámbito. En los últimos años se han publicado diversas aproximaciones de este trabajo [1]. Cierta cantidad de artículos han tratado de implementar clústers de big data basados en hardware de bajo coste y bajo consumo de energía [2]. Sin embargo, en los trabajos previos no se exploraba la optimización de la potencia computacional teniendo en cuenta el consumo eléctrico. La curva de Pareto pretende dar un repositorio de las configuraciones con mejor compromiso entre las dos variables, explorando no solo los límites de aprovechamiento de recursos computacionales, sino de rendimiento energético.

1.2. Objetivos.

El principal objetivo de este trabajo es desarrollar un clúster que sirva como una prueba de concepto que recoja todas las herramientas que debería emplear un sistema para trabajar en computación de altas prestaciones. La puesta en marcha de un clúster de este tipo requiere un plan de trabajo que incluya una serie de pasos técnicos que van desde el ensamblaje del hardware hasta la configuración de un software específico que soporte los protocolos de distribución del trabajo empleados en HPC. A continuación, se describen los objetivos específicos que se pretenden abordar con este trabajo.

1. Análisis de costes y viabilidad del clúster.
2. Aplicación de los conceptos de paralelización y escalabilidad a la ejecución de benchmarks específicos.
3. Control sobre la creación de procesos e hilos en la paralelización gracias al aprendizaje de MPI y OpenMP.
4. Análisis del rendimiento del clúster para sacar el máximo provecho a las prestaciones que es capaz de ofrecer siguiendo unas métricas de ahorro energético y potencia computacional frente al cálculo de una curva de Pareto que optimice el espacio de soluciones.

5. Análisis de escalabilidad de un clúster formado por Raspberry Pi con la posibilidad de escalar el clúster a un mayor número de nodos de cómputo.

Del último de los objetivos surge un trabajo futuro, que no se ha podido llevar a cabo, basado en el ensamblaje de un clúster con un significativamente mayor número de nodos de cómputo. Todos los objetivos específicos presentados están inherentemente asociados al plan de trabajo que se ha seguido durante el proceso de puesta en marcha del clúster. El cronograma de la Figura 1 relaciona las etapas del plan de trabajo con los objetivos que se pretenden abordar en cada una de ellas. Cada proceso a seguir en el plan de trabajo irá asociado a uno o más objetivos dependiendo de su complejidad.

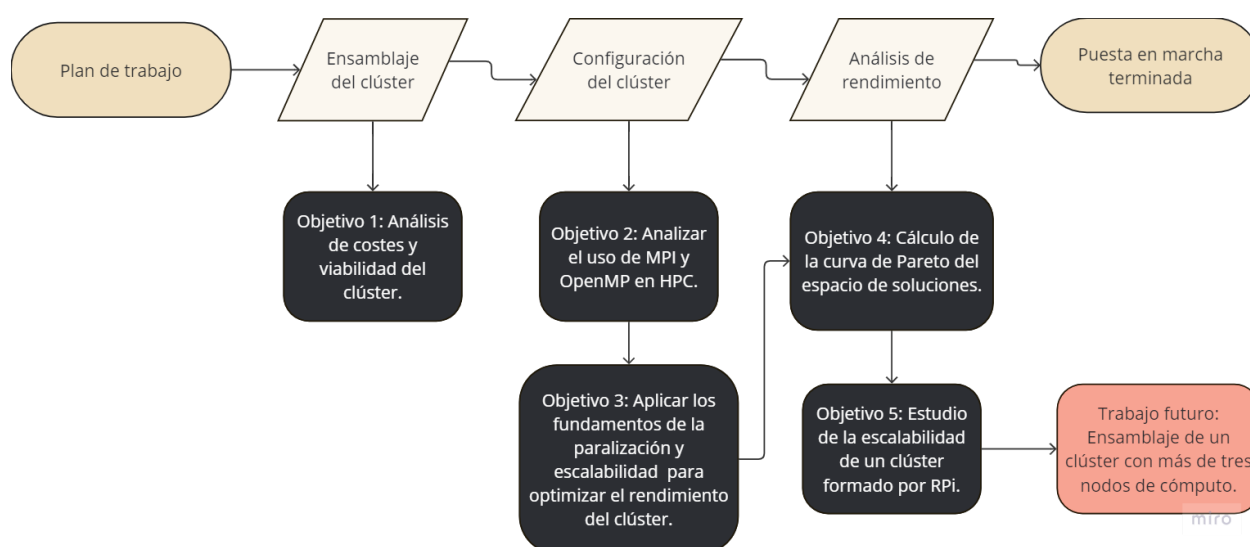


Figura 1: Objetivos específicos que se pretenden abordar dentro de cada etapa del plan de trabajo. Fuente: elaboración propia.

Uno de los objetivos específicos que se presenta es la realización de un estudio acerca del rendimiento que ofrece el clúster. Para abordar el estudio se propone realizar distintas pruebas enfocadas en analizar aspectos del clúster como el aumento de su capacidad de cómputo o el ahorro energético derivados de utilizar aquellas configuraciones que resulten más favorables. La potencia computacional y el consumo energético del clúster constituirán las métricas empleadas en la evaluación del clúster durante las pruebas de caracterización.

1.3. Plan de trabajo

Para cumplir con los objetivos, el plan de trabajo a seguir se ha dividido en tres etapas:

- Montaje del clúster.
 - Compra y/o adquisición del material necesario.
 - Conexión y alimentación de las placas.
 - Configuración de una red de conexión entre las placas.
- Configuración del software.
 - Creación de una imagen ejecutable del sistema operativo para los nodos de cómputo y el *front-end*.
 - Instalación de un sistema de ficheros compartidos a todos los nodos del clúster.
 - Configuración DHCP de cada nodo.
 - Instalación de las herramientas MPI y OpenMP.
- Evaluación del rendimiento del sistema.
 - Definición de las métricas empleadas para evaluar el rendimiento del clúster considerando la eficiencia energética y la potencia de cómputo como los objetivos a alcanzar.
 - Ejecución de un benchmark definido que permita cuantificar el rendimiento del clúster.

2. Estado del arte.

El término Computación de Alto Rendimiento (HPC) se refiere a la solución de problemas científicos que son computacionalmente complejos y costosos, tanto en términos de recursos como de tiempo, mediante el uso de procesamiento paralelo y distribuido. El rendimiento requerido en HPC demanda una cantidad de nodos de alta capacidad, que puede alcanzar incluso millares, operando de manera simultánea, con un gran ancho de banda y baja latencia en la red de comunicación entre ellos. Esta configuración permite obtener resultados de cálculos complejos en un tiempo significativamente menor que el necesario para realizar los mismos cálculos en ordenadores convencionales.

La HPC se desarrolla en supercomputadores. Naturalmente, la capacidad de estos supercomputadores varía según el presupuesto destinado tanto a su configuración como a su mantenimiento, incluyendo el consumo energético, la administración y la actualización de componentes. Estas diferencias presupuestarias se reflejan en variaciones significativas en el rendimiento de los distintos

supercomputadores.

Desde 1993, con el objetivo de proporcionar a la comunidad científica una referencia para establecer colaboraciones y tener una visión actualizada del panorama global de la HPC, se elabora una lista que clasifica los 500 sistemas computacionales más potentes a nivel mundial. Esta lista, conocida como TOP500 [3], se actualiza bienalmente en junio y noviembre, y la participación en ella es voluntaria. Para determinar qué supercomputadores pueden formar parte de la clasificación del TOP500 y en qué posición se encuentran, se emplea un benchmark conocido como LINPACK [4] [5] para evaluar su rendimiento en términos de Flops. A día de hoy el top 3 de ordenadores más potentes del mundo (Frontier[6], Aurora[7], Eagle[8]) están ubicados en Estados Unidos y operan todos con distribuciones del SO Linux. Entre los tres suman una capacidad de computo de 2.7792 PFlops/s (con 1.206 PFlops/s, 1.012 PFlops/s y 561,20 PFlops/s de potencia cada uno). En abril de 2005, el Dr. Wu-chun Feng presentó la idea de crear una lista que clasificara los superordenadores más potentes según su eficiencia energética, en paralelo a la lista TOP500. La reintroducción de esta idea en 2006 generó un aumento del interés en la comunidad por su implementación, y en 2007 se publicó la primera lista Green500 [9]. En esta lista, los superordenadores se clasifican según su eficiencia energética, medida en GFlops/vatio, lo que indica cuántas operaciones se realizan por cada vatio consumido.

Los avances en HPC han sido impulsados por mejoras en hardware, software y técnicas algorítmicas. Procesadores más rápidos, mayores capacidades de memoria y sistemas de almacenamiento más eficientes han permitido aumentar la velocidad y la eficiencia de los cálculos. Paralelamente, el desarrollo de software optimizado para HPC y técnicas de programación paralela han mejorado la capacidad de los sistemas para manejar tareas complejas.

2.1. Uso de Raspberry Pi en HPC.

La Raspberry Pi es un computador de placa única, de bajo coste y tamaño reducido, que ha ganado gran popularidad en la educación, así como en la experimentación y el desarrollo de prototipos. Aunque no fue diseñada originalmente para HPC, su bajo coste y facilidad de uso han inspirado a investigadores y educadores a explorar su potencial en este campo.

La evolución de las placas Raspberry Pi, desde la Raspberry Pi 1 lanzada en febrero de 2012 hasta la Raspberry Pi 5 en octubre de 2023, muestra un notable avance en rendimiento, conectividad y capacidad de memoria. La Raspberry Pi 1 introdujo la computación asequible con un procesador ARM11 de 700 MHz y 512MB de RAM. Posteriormente, la irrupción de la Raspberry Pi 2 en 2015 mejoró significativamente las prestaciones del primer modelo con un procesador de

cuatro núcleos ARM Cortex-A7 y 1GB de RAM. La Raspberry Pi 3, lanzada en 2016, añadió conectividad Wi-Fi y Bluetooth integradas y adoptó un procesador de 64 bits a 1,2 GHz, lo que marcó un importante salto tecnológico. En 2019, la Raspberry Pi 4 ya incluía un procesador de cuatro núcleos ARM Cortex-A72 a 1,5 GHz, con opciones de hasta 8GB de RAM, y soporte para doble pantalla 4K, USB 3.0, y mejor conectividad. Finalmente, la Raspberry Pi 5 en 2023 incorporó un procesador ARM Cortex-A76 a 2.4 GHz, con opciones de 4GB y 8GB de RAM, junto con mejoras en gráficos, almacenamiento y conectividad, consolidándose como una opción potente para aplicaciones más exigentes.

Se han creado múltiples clústers usando Raspberry Pi, como el que se describe en el artículo “Low-cost big data cluster using Apache Hadoop and Raspberry Pi. A complete guide” [10], un clúster de 8 Raspberry Pi desarrollado por la Federal University of Sergipe. En este trabajo se desarrolló un clúster de Raspberry Pi 3 dedicado a big data. Otros artículos [11], por ejemplo, exploran el desarrollo de HPC de bajo coste empleando arquitecturas asimétricas ARM.

Los desafíos que presenta desempeñar computación de altas prestaciones con este tipo de placas son principalmente:

- Limitaciones de hardware: las capacidades de procesamiento y memoria de la Raspberry Pi son limitadas en comparación con los nodos de computación tradicionales en HPC. Esto restringe la complejidad y el tamaño de los problemas que pueden abordarse.
- Escalabilidad: aunque es posible construir clústers grandes con Raspberry Pi, gestionar y mantener estos sistemas puede ser un desafío, especialmente cuando se requiere una comunicación eficiente entre muchos nodos.
- Rendimiento: el rendimiento de los clústers de Raspberry Pi es significativamente menor que el de los sistemas HPC habituales, lo que limita su uso a aplicaciones y experimentos a pequeña escala.

2.2. Concurrency y paralelismo.

Ligados a los términos procesador lógico y núcleo, a nivel software encontramos los conceptos de proceso e hilo. Un proceso es una entidad independiente de ejecución, gestionada por el sistema operativo, que se ejecuta en un momento dado en un núcleo del procesador. Un sistema operativo (SO), por otra parte, es un conjunto de programas y servicios que actúan como intermediarios entre el hardware de un sistema informático y los usuarios, proporcionando una interfaz que permite la ejecución de aplicaciones y la gestión de recursos de hardware de manera eficiente y segura. El

SO tiene la responsabilidad de crear y destruir procesos, así como de planificar su ejecución y gestionar la comunicación entre ellos. Es crucial no confundir los términos “programa” y “proceso”; un programa puede ejecutarse en un único proceso, pero también puede estar compuesto de varios procesos que se ejecutan simultáneamente, aunque no necesariamente comiencen y terminen al mismo tiempo. Este fenómeno se conoce como concurrencia.

Los hilos, también referidos como hebras (*threads* en inglés), son entidades de ejecución independientes que existen dentro de los procesos. Como tal, comparten el código, los datos almacenados en memoria y los recursos del SO asignados al proceso, como los archivos abiertos. Esta característica de los hilos facilita su comunicación, ya que pueden comunicarse utilizando el espacio de memoria compartido sin necesidad de intervención del SO. Además, la creación de un nuevo hilo dentro de un proceso existente, así como su terminación o su conmutación en un determinado núcleo del procesador, puede realizarse de manera mucho más rápida que la de los procesos, dado que la cantidad de información específica que el SO debe gestionar es significativamente menor.

Los hilos abren la posibilidad de paralelizar la ejecución de un programa, o proceso, entre los distintos núcleos de un procesador. Al compartir espacio de memoria, no es necesario un protocolo de mensajería que penalice innecesariamente el rendimiento de la máquina. Los procesos están más comúnmente asociados con un modelo de memoria distribuida donde cada uno tiene su propio espacio de memoria independiente. Aunque se pueden emplear procesos en sistemas de memoria compartida, el punto principal de utilizar procesos es que no requieren ni dependen de un espacio de memoria común para la comunicación entre ellos. Esto los hace muy útiles en clústers que integran gran cantidad de procesadores lógicos con espacios de memoria independientes. Por el contrario, el alcance de los hilos está limitado al espacio de memoria compartida en el que trabajen. Un procesador puede procesar al mismo tiempo el mismo número de hilos que el número de cores que tiene como máximo. De esta forma, si un procesador tiene cuatro cores, solo podrá ejecutar cuatro hilos a la vez. Este es el caso de las placas Raspberry 4, donde el número de hilos por placa está limitado como máximo a cuatro.

El paralelismo es un caso específico de concurrencia en el que se plantea un escenario donde la resolución de un problema puede acelerarse mediante la metodología “divide y vencerás”. El paralelismo lo que hace es tomar el problema inicial, dividir el problema en fracciones más pequeñas, y luego procesar cada fracción de forma concurrente, aprovechando al máximo la capacidad del procesador para resolver el problema. La principal diferencia del paralelismo contra la concurrencia es que, en el paralelismo, todos los procesos concurrentes están íntimamente relacionados al resolver el mismo problema, de tal forma que el resultado de los demás procesos afecta al resultado final. De este modo, cada proceso ejecuta una parte del problema, y existe una etapa final en la que

los resultados parciales calculados por cada proceso se combinan para obtener el resultado final.

Dado que un computador puede trabajar con varios procesos e hilos, existen varios escenarios posibles. El más común en la actualidad es el uso de procesos multihilo, lo que permite una mayor flexibilidad y control para mejorar el rendimiento de las aplicaciones. Este escenario es el que se explorará durante los ensayos de evaluación del rendimiento que es capaz de alcanzar el clúster. Cada nodo de cómputo podrá procesar de uno a cuatro procesos paralelamente. Dependiendo del número de procesos que se estén ejecutando, la cantidad de núcleos dedicados a cada uno será distinta. Teniendo en cuenta que cada nodo de cómputo del clúster tiene cuatro núcleos, si solo se ejecuta un proceso este podrá paralelizarse, como máximo, con cuatro hilos (uno por cada núcleo).

En línea con todo esto, se han creado modelos de programación paralela cuya clasificación se basa en la arquitectura de la memoria subyacente: compartida o distribuida. OpenMP y MPI son los modelos de programación más representativos de estos modelos.

2.3. Modelos de programación paralela.

Uno de los principales retos a los que se enfrenta la HPC es el desarrollo de algoritmos eficientes que aprovechen el potencial que ofrecen las actuales arquitecturas paralelas, como es el caso de ordenadores multicore o de las unidades de procesamiento de gráficos (o GPU). Para ello es necesario poder descomponer los problemas en partes independientes (procesos) cuyas instrucciones puedan ejecutarse de forma simultánea por varios elementos de proceso (hilos).

OpenMP permite especificar el número de hilos que se quieren crear para un proceso. En caso de no indicar un valor concreto, el sistema adopta automáticamente un único core físico en cada una de las placas. Está permitido especificar un número de hilos mayor que el número de unidades de procesamiento disponibles en el hardware. En este caso se puede optar por permitir que el SO sea el encargado de gestionar el encolado y posterior asignación a un procesador, o bien enlazar explícitamente cada thread OpenMP a una unidad de procesamiento física, método que se conoce como thread affinity y que se llevará a cabo durante las evaluaciones de rendimiento.

MPI, por el contrario, es un entorno de programación paralela en el que no se maneja información compartida en memoria, sino que se basa en el intercambio de mensajes para la comunicación entre procesos. MPI es más comúnmente asociado con un modelo de memoria distribuida donde cada proceso tiene su propia memoria independiente.

Este método de programación paralela presenta portabilidad a una gran variedad de sistemas,

desde máquinas con memoria distribuida hasta una red de estaciones de trabajo. A cada elemento de procesamiento se le asigna un número de identificación único, o rango. Todos los procesadores tienen una copia del mismo programa, pero cada proceso ejecutará distintas sentencias del programa por bifurcaciones introducidas basadas en el rango del proceso.

2.4. Intra-nodo: OpenMP.

Para optimizar el aprovechamiento de todos los recursos hardware que ofrece cada procesador lógico existen metodologías que permiten trabajar con los recursos que ofrece el clúster a nivel de nodo como la que aquí se presenta: OpenMP [12]. Este software dedicado permite aprovechar los recursos que ofrece una CPU empleando para un determinado proceso (o tarea) desde uno hasta el máximo de cores físicos que contiene el procesador mediante la creación de entidades denominadas hilos o threads. Cada hilo se asociará a un core físico, siempre que el número de hilos dedicados a la ejecución no supere la cantidad de cores físicos del procesador.

Este paradigma se basa en que varios threads (hilos de ejecución dentro de un proceso) puedan dedicarse conjuntamente a la misma tarea, pero sobre un conjunto de datos distintos ubicados en la memoria del ordenador, que es compartida por todos ellos. Este tipo de software solo es capaz de trabajar con entidades que compartan espacio de memoria, lo que limita su aplicabilidad a un único procesador.

El fichero de cabeceras `omp.h`, de la biblioteca OpenMP, incluye todas las funcionalidades de OpenMP para un programa escrito en lenguaje C. Las directivas `#pragma` en el código indican al compilador como debe ejecutarse el código a nivel de núcleo. A nivel de Raspberry Pi, tras el uso de OpenMP se puede llegar a conseguir activar todos los núcleos disponibles en una placa (cada placa solo cuenta con un procesador). Algunos ejemplos de directivas son: La directiva `#pragma omp parallel` que crea una región paralela donde múltiples threads ejecutan el mismo bloque de código de forma concurrente, compartiendo el trabajo. La directiva `#pragma omp parallel for` combina la creación de una región paralela con la distribución de las iteraciones de un bucle `for` entre los threads disponibles, permitiendo una ejecución paralela eficiente de las iteraciones del bucle.

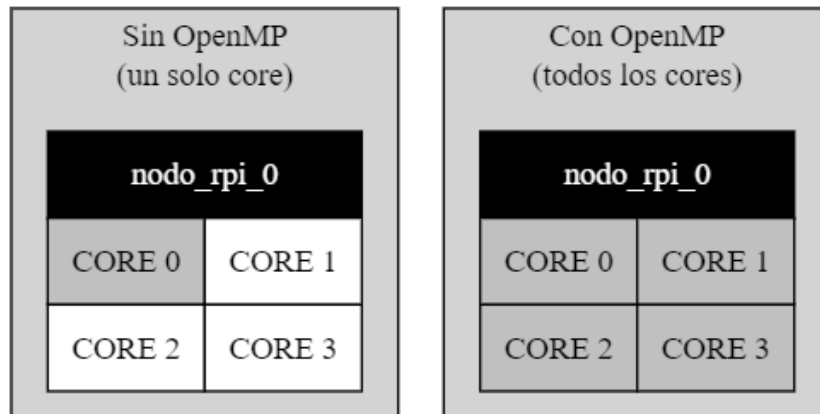


Figura 2: Esquema gráfico de los efectos sobre la cantidad de recursos de la CPU dedicados a la ejecución tras el uso de OpenMP. Fuente: elaboración propia.

La Figura 2 muestra gráficamente el aumento de recursos computacionales que dedica la CPU cuando se paraleliza un proceso en varios hilos mediante el uso de OpenMP.

2.5. Inter-nodo: MPI.

Más allá de que dentro de un nodo todos sus cores físicos puedan trabajar simultáneamente empleando sus recursos en una única tarea (o proceso), para aprovechar la escalabilidad que ofrece un clúster es necesario un protocolo que comunique los nodos entre sí. Esta visión escala el paralelismo a un número de procesos multihilo que exceda la cantidad de núcleos que ofrece un solo procesador. En definitiva, el uso de un protocolo asociado a un modelo de memoria distribuida que no requiera de la memoria compartida para la comunicación entre procesos permite dividir un programa en procesos que abarquen varios nodos de cómputo independientes.

En caso de necesitar una ejecución distribuida, coordinar todos los nodos del sistema exige un protocolo de comunicación entre ellos. La interfaz de paso de mensajes empleada en el mundo de la HPC es el estándar MPI. El estándar MPI [13] es una especificación para la programación paralela que permite la comunicación y sincronización entre múltiples procesos en un sistema distribuido. MPI proporciona un conjunto de funciones para enviar y recibir mensajes, gestionar la ejecución de procesos y coordinar tareas entre ellos, facilitando el desarrollo de aplicaciones paralelas eficientes y escalables. Al funcionar mediante una interfaz de mensajería, MPI no requiere ejecutarse dentro de un sistema con memoria compartida por lo que su aplicabilidad trasciende a una sola CPU. MPI abre las posibilidades de paralelizar un programa entre los distintos nodos de un clúster.

El protocolo de mensajería MPI acarrea consigo una caída en el rendimiento del clúster derivada de los recursos que consume inherentemente el traspaso de mensajes para la comunicación entre los distintos procesos. En la Sección 5 se analiza la caída en el rendimiento que supone emplear un proceso por cada núcleo del clúster, frente a utilizar un solo proceso por cada nodo de cómputo dedicando a su ejecución el máximo número de hilos permitidos por cada procesador. La Tabla 7 recoge que sobrecargar al sistema con procesos innecesarios en lugar de aprovechar el ahorro que supone suprimir el protocolo de mensajería MPI por hilos dentro de cada procesador deriva en una caída de un 5,61 % en el rendimiento final del clúster.

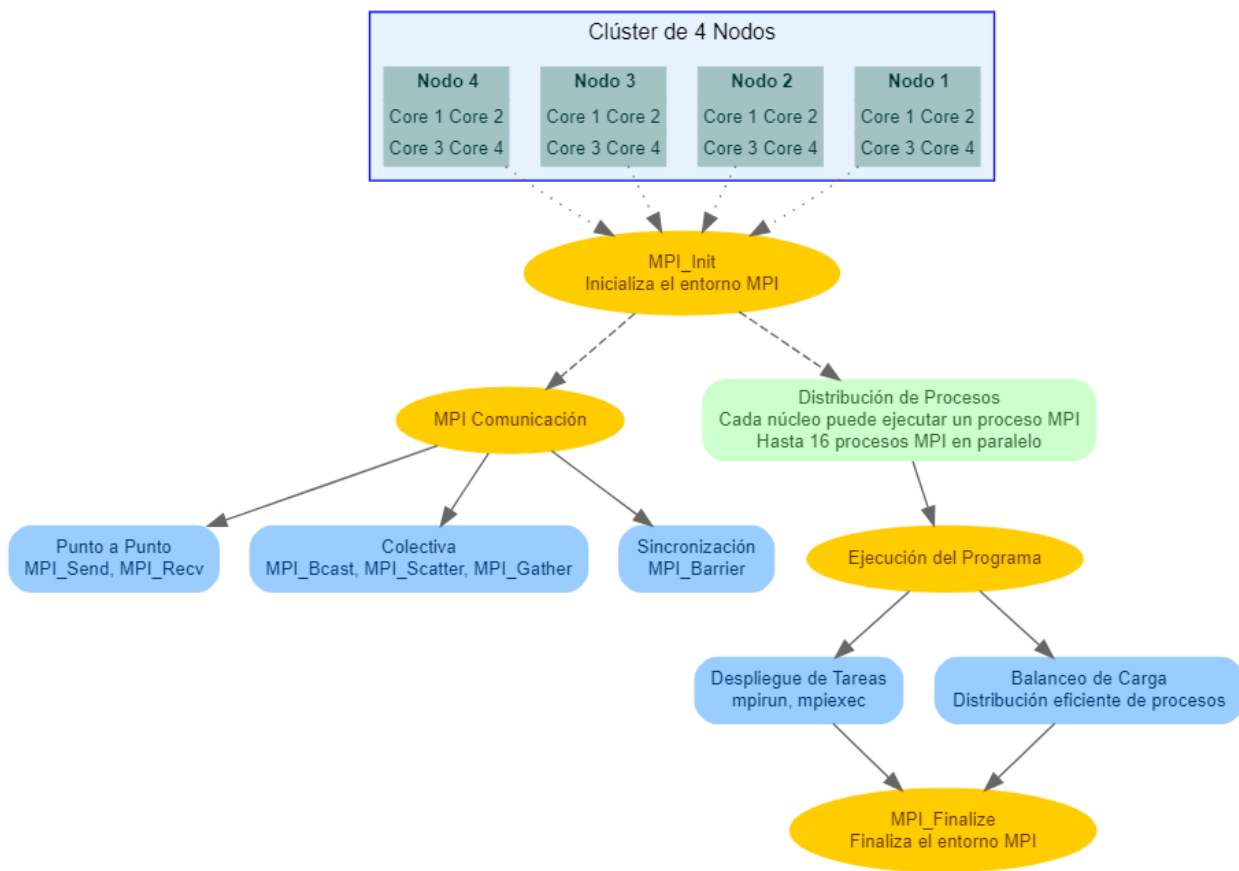


Figura 3: Esquema de comunicaciones y funcionalidades de MPI en el clúster desarrollado en este TFM de cuatro nodos Raspberry Pi, con cuatro cores cada uno. Fuente: elaboración propia.

Gracias al uso de esta interfaz de mensajería los nodos pueden comunicarse entre sí. Cada uno accede a direcciones de memoria no compartidas con el resto, pero puede transferir información a los demás. Mientras que en OpenMP se hablaba de hilos, en el caso de MPI se hace referencia a procesos. Cada entidad de ejecución dispondrá de su propio espacio de memoria, sin importar que se trate de procesos en el mismo o distintos nodos.

La imagen de la Figura 3 proporciona una representación del flujo de trabajo y las operaciones comunes dentro de un clúster de cómputo de 4 nodos utilizando MPI. La parte superior de la imagen representa un clúster con 4 nodos, cada uno con 4 núcleos (cores). Estos núcleos pueden ejecutar procesos en paralelo. Las líneas de puntos conectan el clúster de nodos con la inicialización de MPI (MPI_Init). Esto sugiere que MPI_Init se encarga de iniciar y establecer el entorno MPI en el clúster, permitiendo que los procesos MPI se distribuyan y ejecuten en los diferentes nodos y núcleos. Es la primera etapa en el flujo donde se inicializa el entorno de MPI. Esto es esencial para preparar el entorno en el que se ejecutarán los procesos en paralelo en los diferentes núcleos. Una vez inicializado el entorno MPI, los procesos se distribuyen a través de los núcleos disponibles en el clúster. Hasta 16 procesos MPI pueden ejecutarse en paralelo. Durante la ejecución del programa, se gestionan tareas como el despliegue de tareas (mpirun, mpiexec) y el balanceo de carga para asegurar que los procesos se distribuyan eficientemente entre los recursos disponibles. Al final del flujo, MPI_Finalize se encarga de cerrar el entorno MPI, concluyendo la ejecución de los procesos paralelos.

3. Montaje del clúster.

El ensamblaje del clúster requiere un rack que proporcione una estructura fija sobre la que colocar permanentemente los elementos que lo conforman. La Figura 5 muestra la mecánica implicada en la estructuración del clúster. El rack cuenta con un total de cinco paneles para separar las placas y poder aplicarlas una sobre otra. Las dimensiones externas del rack son de 115 x 123 x 181 mm. En total, se necesitan 16 tornillos de montaje, que funcionen como separadores entre los paneles, y 16 tornillos avellanados que fijen la posición de las placas Raspberry Pi a la estructura.

La Figura 5 muestra el hardware requerido para el ensamblaje previo al conexionado de la red de comunicación. El clúster que se presenta en este trabajo está compuesto por cuatro placas Raspberry Pi que se comunican a través de una red interna y local, mediante cables Ethernet. El modelo de placa utilizado se detalla en la Tabla 1. Cada placa necesita una fuente de alimentación de 5 V y 3 A y un sistema de refrigeración individualizado [14]. Este requisito no es muy específico, dependerá de las exigencias a las que se vaya a someter a la máquina. En este caso particular se ha optado por unos ventiladores alimentados a 5 V dedicados a la refrigeración de este tipo de sistemas empotrados [15]. El consumo del clúster rondará los 60 W cuando se encuentre funcionando activamente y los 11 W cuando los nodos estén en reposo.

La colaboración entre los nodos depende directamente de la red de comunicación con la que se haya conectado el clúster. Para esta aplicación se ha optado por una topología de tipo estrella, en la que todos los nodos se comunican a través de un conmutador (o switch). Todos los nodos están

4 x Raspberry Pi Model B	
CPU	1× ARM1176JZF-S 700 MHz
RAM	2 GB LPDDR4-3200 SDRAM
USB	2x USB 2.0
Storage	32 GB SD Class 10

Tabla 1: Especificaciones de la placa Raspberry Pi 4 Model B empleada en el clúster.

conectados al conmutador, de modo que si alguno de ellos falla, la comunicación entre el resto de nodos no se ve afectada. El componente más importante de la red es el conmutador, que funciona como intermediario para todas las conexiones entre los nodos. En caso de fallo en el conmutador, la red quedaría inhabilitada completamente.

La Figura 4 ilustra algunas de las posibles topologías que podría tener la red de comunicación. La topología totalmente conexa, por ejemplo, conecta todos los nodos entre sí sin necesidad de intermediarios para ninguna conexión. Esto hace que sea la topología con mayor ancho de banda, pero también aquella que requiere un cableado más complejo.

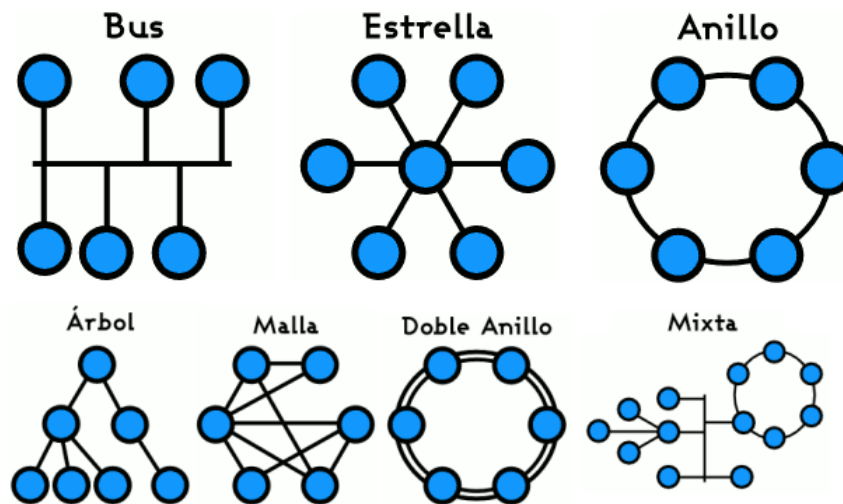


Figura 4: Ejemplos de topologías en redes de comunicación. Fuente: <https://247tecnologia.com/topologia-de-red-tipos-caracteristicas/>

En computadores como la Raspberry Pi, la memoria principal es de lectura y escritura, y es volátil. Esto significa que los datos almacenados se pierden si la memoria se desconecta de la fuente de energía. En los nodos, esta memoria principal, o RAM, es donde se cargan las instrucciones y datos que el procesador utilizará. La memoria principal disponible no equivale a la memoria total en el nodo, ya que el SO utiliza y reserva parte de esta memoria para sus operaciones internas. Esta reserva será más notable en el nodo que funcione como *front-end*, donde la interfaz gráfica

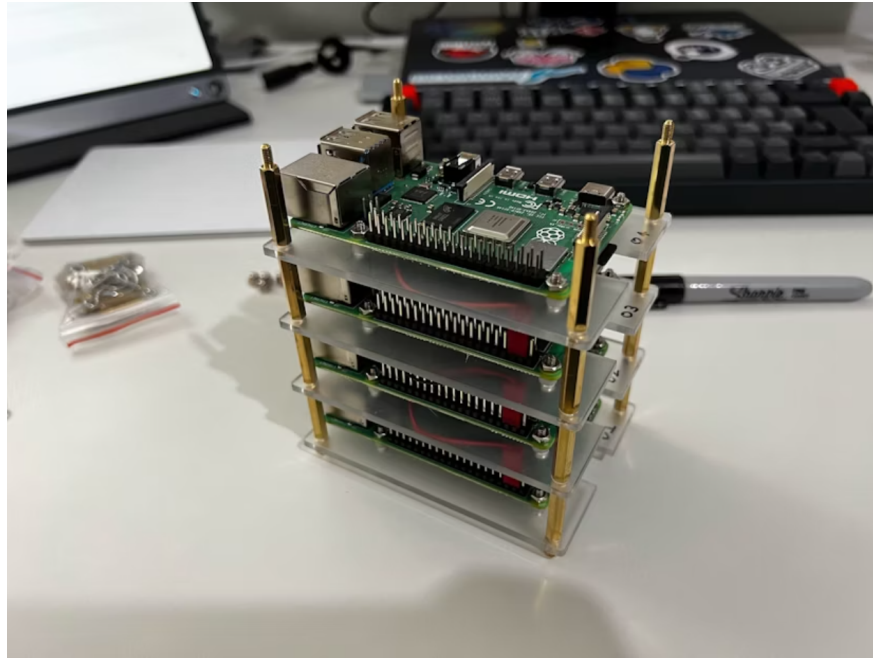


Figura 5: Fotografía del clúster de Raspberry Pi construido.

no está deshabilitada. La memoria principal de la Raspberry Pi es SDRAM dinámica síncrona de bajo consumo energético (LPDDR4 SDRAM) con una capacidad de 2 GB.

El almacenamiento, o memoria secundaria, es un tipo de memoria no volátil necesaria para el funcionamiento adecuado de un ordenador. Este tipo de memoria es más económica, pero tiene un acceso más lento que la memoria principal. Por este motivo, el espacio de almacenamiento suele ser significativamente mayor que el de la memoria principal. En el caso de la Raspberry Pi, y específicamente en este trabajo, se ha optado por utilizar una memoria secundaria de tipo SDHC conectada al Bus SD mediante el zócalo Micro SD incorporado en la tarjeta. En particular, se ha utilizado una tarjeta Micro SDHC de Clase 10, con una capacidad de 32 GB y una velocidad de escritura secuencial de hasta 10 Mbps [16].

3.1. Coste económico.

Los supercomputadores generalmente suponen un coste de hasta cientos de millones de euros (Top 3 del Top500, [6] [7] [8]). El objetivo de este trabajo no es ese, sino fabricar un modelo que incluya la jerarquía de un supercomputador, pero de bajo coste. Las tarjetas Raspberry Pi se crearon con la filosofía de ser dispositivos versátiles y económicos.

El presupuesto aproximado del clúster, sin tener en cuenta los periféricos, se recoge en la Tabla 2. En ella puede verse como por menos de 400 € podrían adquirirse los componentes de un clúster

similar a este. En función del presupuesto del que se disponga, este clúster es totalmente escalable al número de nodos que sean necesarios. Al coste total del clúster habrá que sumar el consumo de energía de electricidad. Se calcula que para este trabajo se han empleado entorno a 35 horas de uso moderado dedicadas a la configuración, con un consumo de unos 25 W y 20 horas a la ejecución de benchmarks para la caracterización del clúster, con un consumo de unos 60 W. Se calcula un gasto derivado del consumo eléctrico [17] de unos 13 € utilizando el precio medio de la electricidad en el mes de Mayo de 2024.

Componente	Cantidad	Precio/ud (€)
Raspberry Pi 4 [18]	4	94,63
Tarjeta microSD 32GB [16]	4	6,99
Fuente de alimentación 5V 3A	4	8,00
Conmutador (SWITCH) [19]	1	14,99
Ventilador de refrigeración [15]	4	2,59
Total:		395 €

Tabla 2: Lista de componentes con sus cantidades y precios.

Esta aplicación ha utilizado un conmutador modelo TP-Link LS105G de 5 puertos como piedra angular de la red de comunicación. El fabricante asegura que presenta un ancho de banda que puede llegar hasta los 1Gb/s. Esta velocidad de red asegura el máximo aprovechamiento de los recursos que ofrecen las tarjetas Raspberry Pi 4, que están dotadas de una interfaz de red con un ancho de banda de las mismas prestaciones.

Este supercomputador es fácilmente escalable a un mayor número de Raspberry Pi. Cada placa añadida supondría un coste de compra de unos 95 € y un aumento del consumo eléctrico de unos 8W, de media. A medida que el número de nodos aumente, el clúster cada vez competirá en capacidad de cómputo con laptops de mayor coste.

4. Configuración del clúster.

Un clúster dedicado a la HPC requiere un fuerte soporte software especializado en paralelizar la carga computacional a la que se somete. En este trabajo se empleará una red de comunicación para configurar un sistema de ficheros compartidos entre todos los nodos, simulando así el funcionamiento de un clúster de supercomputación. Los nodos pueden, o no, compartir espacio de almacenamiento. En ocasiones los clústers emplean un sistema de ficheros compartido, como este

caso. Otras veces, utilizan memoria distribuida, en la que no todo el espacio de almacenamiento está compartido entre todos los nodos. Aprovechar todos los recursos de los que dispone un clúster requiere, en estas ocasiones, un protocolo de mensajería que comunique los procesadores que trabajan con espacios de memoria independientes entre sí.

El primer paso tras ensamblar las Raspberry Pi en el rack será instalar el SO que actuará como intermediario entre el usuario y los recursos hardware del clúster. En el contexto de la HPC, los sistemas operativos desempeñan un papel crucial para maximizar la eficiencia, escalabilidad y rendimiento de los sistemas de supercomputación. La Figura 6 muestra un diagrama de flujo que

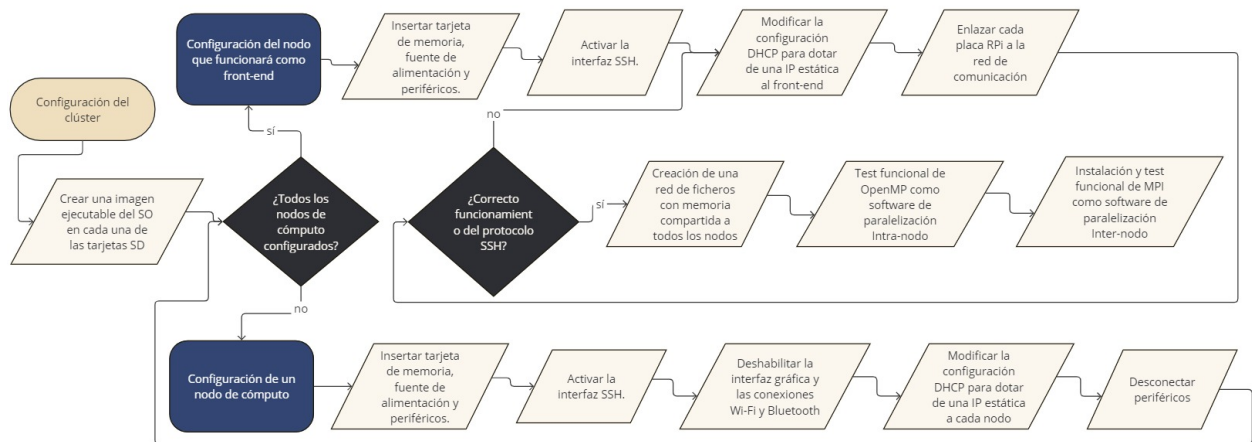


Figura 6: Diagrama de flujo que describe los procesos necesarios para la configuración del clúster. Fuente: elaboración propia.

contiene los pasos a seguir para la puesta a en marcha del software utilizado en el clúster. Este diagrama exhibe una perspectiva de alto nivel de todas los pasos a seguir durante la configuración.

4.1. Creación de una imagen ejecutable de la tarjeta SD.

Al estar trabajando con Raspberry Pi, el SO que se instalará en las tarjetas será el Raspberry Pi OS. Este SO fue desarrollado por The Raspberry Pi Foundation (antes conocido con Raspbian [20]) y es un software de código abierto que puede encontrarse en la bibliografía [20]. La versión *desktop* de este SO no ocupa más de 1,2 GB.

El proceso de creación de una tarjeta SD ejecutable que contenga el SO es sencillo. Existen programas como RaspberryPiImager [21] desde los que se puede descargar una imagen de gran variedad de SO compatibles con Raspberry Pi. Tras este proceso, una vez que la imagen del SO quede grabada, cuando se introduzca la tarjeta SD en la Raspberry se iniciará la ejecución del SO instalado.

4.2. Puesta en marcha de cada nodo del clúster.

Al iniciar por primera vez cada nodo, se repetirán los siguientes pasos:

1. Conectar los periféricos a una de las tarjetas que no ha sido configurada (para la configuración inicial de cada nodo, habrá que conectar los periféricos a cada una de las Raspberry Pi).
2. Insertar las tarjetas de memoria y conectar la fuente de alimentación (5 V y 3 A) a cada placa Raspberry Pi.

En lo sucesivo, una vez configurados todos los nodos de cómputo, se accederá a ellos mediante el protocolo SSH directamente desde el *front-end*. De esta manera se evita tener que utilizar periféricos para controlar los nodos de cómputo. SSH es un protocolo de red destinado principalmente a la conexión con máquinas a las que se accede por línea de comandos. Es imprescindible que los nodos tengan una dirección IP estática dentro de la red interna que los comunica. De no ser así sería imposible comunicarnos remotamente desde un nodo a otro mediante el protocolo SSH. Para conseguirlo será necesario modificar la configuración DHCP que, por defecto, tienen asignados los nodos. El protocolo DHCP es un estándar de red utilizado para asignar automáticamente direcciones IP y otros parámetros de configuración necesarios a dispositivos que se conectan a una red. Esta configuración juega un papel fundamental en la asignación de direcciones IP dentro de una red. En lugar de que un administrador de red asigne manualmente una dirección IP a cada dispositivo, el servidor DHCP gestiona y distribuye estas direcciones de manera automática.

Las siguientes secciones describen los pasos a seguir para activar la interfaz SSH y configurar el protocolo DHCP en todos los nodos de cómputo para que sean accesibles desde el *front-end*.

Nodos de cómputo.

Como los nodos de cómputo solo tendrán conexión con la red local de comunicación privada, la conexión Wi-Fi y Bluetooth puede desactivarse. Además, para destinar la mayor cantidad posible de recursos al cálculo, se recomienda deshabilitar también la interfaz de usuario de todos estos nodos. En la pestaña *Interfaces* será necesario habilitar la interfaz SSH para acceder remotamente a la terminal de cualquier nodo remotamente desde el *front-end*.

Para asignar una dirección IP estática dentro de la red interna a cada nodo será necesario modificar la configuración DHCP que, por defecto, tienen asignados los nodos. A cada nodo se le asignará una de las direcciones IP estáticas que aparecen en la Tabla 3.

La primera columna de la tabla contiene los indicadores que se utilizarán para identificar, de aquí en adelante, cada nodo del clúster, para evitar tener que hacer referencia a las direcciones IP de cada uno. Para modificar la configuración DHCP, que habilita la configuración automática del

Nodo	Tipo de nodo	IP asignada
nodo0	<i>front-end</i>	192.168.0.1/24
nodo1	Cómputo	192.168.0.2/24
nodo2	Cómputo	192.168.0.3/24
nodo3	Cómputo	192.168.0.4/24

Tabla 3: IP estática para cada nodo.

direccionamiento IP hay que editar el fichero al que accede el SO para configurar la red. El SO instalado en las placas para este trabajo accede al siguiente fichero. Este deberá editarse como se muestra a continuación:

Fichero: /etc/systemd/network/eth0.network

```
[ Match ]
Name=eth0

[ Network ]
Address=192.168.1.x/24
Gateway=192.168.0.1
DNS=192.168.0.1
```

Donde la dirección IP 192.168.0.1 (Gateway y DNS) se reserva para la puerta de enlace, que corresponde a la dirección del *front-end*. Cada nodo tendrá su dirección IP por lo que x deberá modificarse adecuadamente para cumplir con las direcciones IP de la Tabla 3.

Una vez editado el archivo habrá que reiniciar la red mediante el demonio del sistema *systemd-networkd*. Finalmente, para que el archivo que contiene las direcciones IP se lea siempre que se arranque el sistema habrá que configurar el reinicio de la red con las siguientes líneas de comandos:

Reiniciar la interfaz de red

```
$ sudo systemctl restart systemd-networkd
$ sudo systemctl enable systemd-networkd-wait-online.service
```

De esta forma este servicio esperará hasta que la red esté completamente configurada antes de continuar con el inicio del sistema. Esto creará un enlace simbólico en el directorio `systemd` que hará que el servicio `systemd-networkd-wait-online` se inicie automáticamente en cada arranque del sistema.

Hasta hace unos años, las versiones de Raspbian (2020) accedían a otra dirección de memoria para configurar el protocolo DHCP. Este método está ya desactualizado pero es posible que en algunos sistemas operativos la configuración de DHCP siga estando localizada en el fichero `/etc/dhcpd.conf`.

Front-end.

El *front-end*, o `nodo0`, es la vía de acceso al clúster, de modo que su puesta en marcha difiere ligeramente de las demás. Este nodo sí que contará con una interfaz gráfica y acceso a Internet, a diferencia de los nodos de cómputo. El acceso a Internet es imprescindible para poder descargar los paquetes de software implicados en las funcionalidades de paralelización y concurrencia que requiere el clúster. El resto de pasos son idénticos a los que se han seguido para el resto de nodos.

Para que el tráfico no se dirija por defecto a la red interna (que conecta los nodos entre sí) y que este nodo siga teniendo acceso a Internet se requiere editar el contenido del fichero `60-gw` como se muestra a continuación:

Fichero: `/lib/dhcpd/dhcpd-hooks/60-gw`

```
route del default gw 192.168.0.1
```

Con el servidor DHCP configurado en todos los nodos del clúster ya se puede proceder con las pruebas de conectividad entre los distintos nodos. Para ello es necesario conectar los elementos del clúster a la red local como se describe en la Figura 7. Cada placa estará conectada mediante un cable Ethernet, que servirá como medio de comunicación con los demás nodos. Todos los nodos estarán interconectados a través del conmutador que controla la red.

Para completar una puesta en marcha de este tipo debe verificarse el buen funcionamiento de la red de comunicación que enlaza los nodos. Para ello debe comprobarse si se puede acceder desde un nodo a otro. La siguiente Consola muestra el comando asociado al protocolo SSH que permite acceder a la dirección IP estática asignada, en este caso, al `nodo1`.

Comandos de Terminal

```
pi@nodo0 $ SSH 192.169.0.2
```

En determinadas circunstancias, un fallo en la conexión de la red, ya sea de origen físico o de configuración, puede generar un error similar a SSH: `connect to host nodo1 port 22: No route to host`, lo que indica la imposibilidad de establecer conexión con esa dirección IP. Este problema puede tener dos causas principales. En caso de que el problema provenga del conexionado o del conmutador, se trataría de un problema de hardware. Para resolverlo es fundamental comprobar que todas las conexiones sean adecuadas y que tanto los cables Ethernet como el conmutador funcionen. En caso de que, tras estas comprobaciones, la red siga fallando es probable que no se haya configurado correctamente el protocolo DHCP. Mediante el comando `ip addr` se pueden visualizar las interfaces de red y sus direcciones IP. Si la configuración DHCP no es la adecuada no se asignará una dirección IP estática a cada nodo del clúster y en el apartado `eth0`, no aparecerá la dirección IP que se configuró anteriormente.

Información de IP del *front-end*

```
pi@nodo0:~ $ ip addr
...
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP
    link/ether dc:a6:32:4d:b7:c4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.1/24 brd 192.168.0.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::dea6:32ff:fe4d:b7c4/64 scope link
        valid_lft forever preferred_lft forever
...
```

4.3. Sistema de ficheros compartidos.

Para simular el funcionamiento de un clúster dedicado a la supercomputación se utilizará la red de interconexión para configurar un sistema de ficheros compartidos entre todos los nodos. Este sistema de ficheros debe ser accesible por todos los nodos. En él, se instalará todo el software relativo a gestión de recursos y aplicaciones para las que se vaya a utilizar el supercomputador.

Para poder mandar los trabajos y que los nodos los pudieran reconocer se creó un directorio compartido en red utilizando NFS [22]. Este directorio debe contar con los permisos adecuados

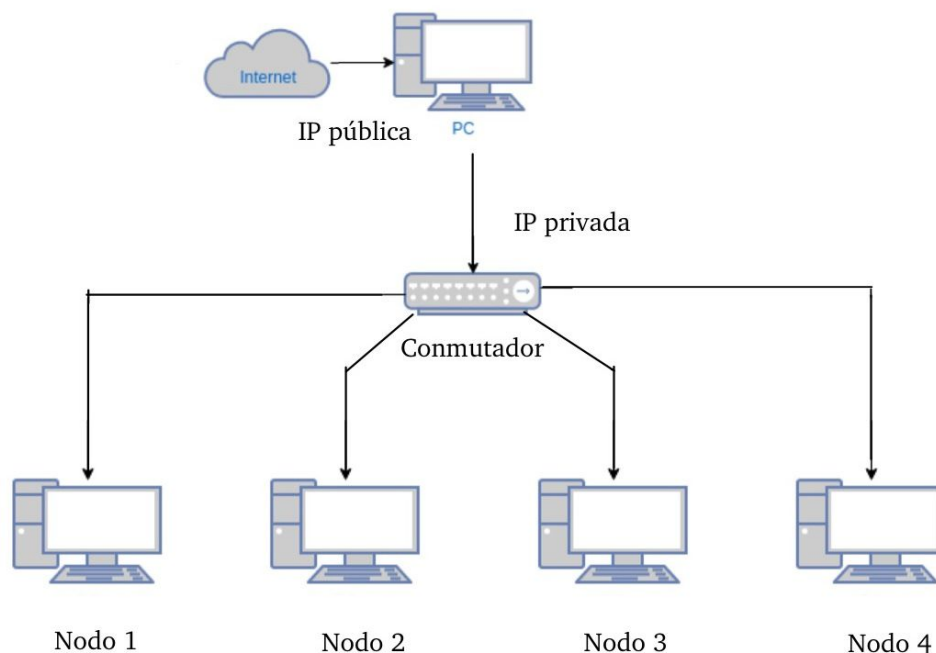


Figura 7: Esquema de la estructura de un clúster con cuatro nodos de cómputo conectados en topología de estrella a través de un switch. Fuente: Elaboración propia.

para que todas las máquinas puedan leer y escribir ficheros en él. Para que el directorio sea funcional es necesario montarlo en cada uno de los nodos. Montar un directorio no es más que asociar un sistema de archivos o un recurso compartido en red a un directorio específico en un sistema (en este caso, en cada nodo), permitiendo que los archivos y carpetas de ese recurso sean accesibles como si fueran parte del sistema de archivos local.

4.4. Modelos de programación paralela.

Para aprovechar al máximo las capacidades de paralelización en sistemas de alto rendimiento, es fundamental instalar y configurar adecuadamente unas herramientas de paralelización. Este clúster dispondrá de los modelos de programación OpenMP y MPI que permitirán dividir los programas en procesos e hilos que agilicen su ejecución. La instalación correcta de OpenMP y MPI permite a los usuarios crear y ejecutar aplicaciones paralelas, optimizando el uso de recursos en arquitecturas tanto de memoria compartida como distribuida. A continuación, se describen los procedimientos para la instalación de MPI, incluyendo las dependencias necesarias, configuraciones específicas y consideraciones importantes para asegurar un entorno de desarrollo robusto y eficiente.

Los pasos a seguir para instalar la biblioteca OpenMPI [23] que implemente el protocolo de mensajería MPI se resumen en descargar el código fuente e instalar la biblioteca en un directorio compartido. Se instala en un directorio compartido para que todos los nodos puedan acceder a él y leer la variable de entorno `PATH` relativa a la ejecución de un programa utilizando MPI. `PATH` es una variable de entorno que especifica el directorio donde el SO debe buscar ejecutables cuando se ejecuta un comando en la terminal. De este modo se podrán ejecutar comandos relativos a MPI en cualquiera de los nodos, ya que todos tienen acceso a las variables de entorno.

En caso de no disponer del código fuente del programa que se va a ejecutar será necesario que este tenga soporte específico para el uso de MPI. De otro modo no será posible dividir la ejecución en varios procesos. El comando para ejecutar un programa utilizando MPI es `mpirun`. Durante la ejecución de un programa empleando MPI será necesario especificar el número de procesos que se van a utilizar mediante la opción “-n”. En caso de querer ejecutar procesos en distintos nodos (o procesadores), habrá que indicar el nombre de cada nodo seguido de la función “-host”. La siguiente consola muestra un ejemplo de paralelización en el que se utilizan cuatro procesos (numerados por “rangos”) repartidos entre 4 procesadores.

Ejecución de un programa dentro de OpenMPI.

```
#El ejecutable hola_mundo_mpi debe de estar en el directorio
#compartido para que todos los nodos puedan acceder.

pi@nodo0:/SHARED $ mpirun -n 4 -host nodo0,nodo1,nodo2,nodo3
./hola_mundo_mpi

Hola mundo desde nodo0, con rango 0 de un total de 4 procesadores.
Hola mundo desde nodo3, con rango 3 de un total de 4 procesadores.
Hola mundo desde nodo2, con rango 2 de un total de 4 procesadores.
Hola mundo desde nodo1, con rango 1 de un total de 4 procesadores.
```

5. Evaluación del rendimiento del clúster.

La escalabilidad de un sistema es una propiedad que refleja su capacidad para responder eficientemente a cambios en su configuración o capacidad. Por ejemplo, en un sistema con dos procesadores, se esperaría que cualquier programa ejecutado en paralelo en dos procesadores tardara la mitad del tiempo que si se ejecutara en un sistema con un solo procesador. Sin embargo, en la práctica, este valor teórico no se cumple debido a diversos factores tanto a nivel de hardware como de software que impiden una escalabilidad lineal perfecta. La escalabilidad es una buena referencia para parametrizar el rendimiento real que presenta un sistema.

Para evaluar el rendimiento del clúster tanto a nivel de potencia de cómputo como de consumo energético se utilizarán dos métricas. Por un lado se analizará el speed up de unas configuraciones frente a otras. Por otro lado, se calculará la eficiencia energética que presenta el sistema para resolver un problema específico a partir de la evolución temporal de la temperatura de la CPU durante las sucesivas ejecuciones.

5.1. Benchmarks empleados para la evaluación del rendimiento del clúster.

El estudio de la capacidad de cómputo del clúster se ha llevado a cabo ejecutando un benchmark que forma parte de los test de ensayo que proporciona el software *open source* de la empresa GROMACS. GROMACS es un paquete de software gratuito dedicado al análisis de resultados y simulación de dinámicas moleculares, de alto rendimiento. Durante los ensayos se ha utilizado el script `int_water_md_verlet_none_pme_switch`, que simula las ecuaciones de movimiento de un sistema con cientos o millones de moléculas de agua interaccionando entre sí. El objetivo del estudio es evaluar como varían las prestaciones del clúster cuando se modifican sus parámetros durante la ejecución. Para ello, un paso imprescindible es establecer una carga computacional estable y repetible para todos los experimentos. Esto es lo que en computación de altas prestaciones se conoce como un benchmark y se utiliza para medir el rendimiento de un sistema en específico y incluso compararlo con el de otro sistema externo.

Cualquier ejecución del software GROMACS requiere de ciertos parámetros de entrada. El programa, para ejecutarse debe tener acceso a algunos parámetros del sistema que se recogen dentro de una serie de ficheros que delimitan y que caracterizan la simulación. En particular, los test que recogen esta información se encuentran dentro de un directorio asignado que da nombre al experimento (en este caso los ficheros se encuentran en el directorio `int_water_md_verlet_none_pme_switch`). El ejecutable se construye mediante el comando `grompp` que preprocesa los archivos de entrada y genera un archivo portátil binario (`.tpr`). La siguiente consola muestra el proceso de ejecución de la simulación con `mdrun` usando MPI y OpenMP:

```
int_water_md_verlet_none_pme_switch.
```

```
# Preprocesar el archivo .tpr
$ gmx_mpi grompp -f system.mdp -c system.gro -p system.top -o em.tpr
# Simulacion
$ mpirun -np <numero de procesos MPI> -host <nodos de computo>
gmx_mpi mdrun -ntomp <numero de cores asignados por proceso MPI>
-v -deffnm em
```

En la segunda línea de comandos de la consola, se recogen entre $\langle \rangle$ ciertas variables que configuran el estado y prestaciones del clúster durante la ejecución. El número de procesos MPI en los que se quiera distribuir la ejecución no tiene limitación hardware en primera instancia, a diferencia del resto de parámetros (como el número de núcleos de los que dispone el clúster) que están limitados por las características del montaje. Se podrá observar, si el código fuente de GROMACS efectivamente soporta MPI, como al paralelizar la ejecución en varios procesos, entre los distintos procesadores del clúster, el tiempo de ejecución se reduce. No siempre aumentar el número de procesos (o hilos) se traduce en una mayor optimización del tiempo. Habrá que entender cómo se paralelizan los procesos MPI entre los núcleos para hallar la configuración optimice el tiempo de ejecución y el consumo del clúster.

El parámetro que acompaña `-ntomp` es el número de núcleos que se asignarán a cada proceso MPI. Teniendo esto en cuenta, para optimizar la paralelización, el producto del número de procesos por el número de hilos destinados a cada uno nunca debe superar la cantidad de núcleos disponibles. Además, hay que tener en cuenta que OpenMP (controlado durante la ejecución por `-ntomp`) solo trabaja a nivel de nodo. Cuando se utilicen varios nodos simultáneamente, será necesario dividir la ejecución en varios procesos MPI (que permite comunicar los nodos entre sí) para poder aprovechar al máximo el potencial disponible del clúster.

5.2. Métricas empleadas para la evaluación.

Speed up

El speed up de un sistema es una variable adimensional que mide la mejora en el rendimiento de un sistema que procesa un problema determinado. Se define como:

$$speedup := \frac{T_1}{T_2} \quad (1)$$

Donde T_1 o “tiempo de referencia” suele ser el tiempo de ejecución del mismo benchmark de la arquitectura con la que estamos comparando al sistema. T_2 ó “tiempo de ejecución” no es más que la latencia del sistema bajo estudio.

El speed up puede definirse también de forma porcentual. Por ejemplo, un speed up de x3 se correlaciona directamente con un aumento de un 200 % en la velocidad de ejecución o con una disminución del 66.7 % en la latencia. Estas métricas auxiliares se definen como:

$$speedup_v(\%) := 100 \cdot \frac{T_1 - T_2}{T_2} = 100 \cdot (speedup - 1) \quad (2)$$

Para el aumento en la velocidad de ejecución.

$$speedup_t(\%) := 100 \cdot \frac{T_1 - T_2}{T_1} = 100 \cdot (1 - \frac{1}{speedup}) \quad (3)$$

Para la disminución en el tiempo de ejecución.

Todas estas definiciones cuantifican la misma métrica: la mejora de rendimiento del sistema. La variable medida para calcular la mejora del rendimiento en este caso será el tiempo de ejecución o latencia de cada configuración durante los experimentos.

El speed up es una variable relativa. Normalmente se calculará en referencia al tiempo de ejecución más longevo dentro de un experimento a fin de evitar influencias ambientales. Es posible que en ocasiones esto pueda no ser así, como en la Tabla 7 donde los speed up están normalizados al peor caso de 3 experimentos distintos.

Ahorro energético.

Otra de las métricas utilizadas para caracterizar el clúster ha sido el ahorro energético normalizado relativo entre unas configuraciones y otras, calculado como el cociente entre el consumo energético de cada una. El ahorro energético no tiene unidades de energía, está normalizado al consumo de la configuración menos óptima en este sentido. El consumo total del sistema durante una ejecución completa se ha calculado como la integral en el tiempo de la diferencia entre la temperatura a la cuarta potencia de la CPU de los tres nodos de cómputo y la temperatura ambiente a la cuarta potencia.

La Ecuación 4 recoge el cálculo de la eficiencia energética relativa normalizada. Donde W_1 y W_2 son los consumos eléctricos asociados a las dos configuraciones que se comparan. Esta métrica permite evaluar cuánta energía se ahorra o se consume adicionalmente al cambiar ciertos parámetros del sistema. Generalmente, en cada experimento se referenciarán todos los ahorros energéticos a la configuración que requiera un consumo más alto (W_2 será común a todos los cálculos dentro de cada experimento). En este punto surge la posibilidad de evaluar una métrica que fusione las ya

presentadas hasta el momento. Para ello se puede emplear la variable de la capacidad de cómputo partida por el consumo eléctrico normalizado a la configuración más desfavorable en este sentido.

$$Eff := 100 \cdot \frac{W_1}{W_2} (\%) \quad (4)$$

Durante los ensayos realizados para evaluar la potencia disipada por el clúster de Raspberry Pi, se ha empleado la Ley de Stefan-Boltzmann como base para los cálculos. Este enfoque se ha adoptado debido a la capacidad de esta ley para proporcionar una estimación precisa de la potencia disipada a través de la radiación térmica.

La Ley de Stefan-Boltzmann se expresa matemáticamente como:

$$P = \epsilon \sigma A (T^4 - T_{\text{amb}}^4)$$

En este contexto, la potencia disipada P está relacionada con la emisividad del objeto ϵ , la constante de Stefan-Boltzmann σ , el área de la superficie del objeto A , la temperatura del objeto T y la temperatura ambiente T_{amb} .

Durante este ensayo se han supuesto constantes las siguientes variables:

- La emisividad del clúster de Raspberry Pi, ϵ , ha sido considerada como un valor constante específico del material de los componentes.
- El área superficial del clúster, A , ha permanecido invariante durante el ensayo.
- La temperatura ambiente, T_{amb} , se ha registrado como constante en 27.3 °C (300.45 K) para todos los experimentos.

Debido a la constancia de estas variables, la ecuación se ha simplificado para expresar la potencia disipada como una función de la temperatura del clúster de Raspberry Pi. Con T_{amb} fija, la ecuación de Stefan-Boltzmann se reescribe para enfatizar la dependencia de P respecto a T :

$$P = \epsilon \sigma A (T^4 - (300,45)^4)$$

Es importante destacar que la temperatura ambiente T_{amb} de 27.3 °C se mantuvo constante a lo largo de todos los ensayos para asegurar la validez de las medidas. Además, la emisividad y el área superficial del clúster también permanecieron invariables, garantizando que cualquier cambio en la potencia disipada estuviera directamente relacionado con la temperatura del clúster. Debido a esto, la Figura 15 expresa como parámetro de consumo energético la integral de: $P/(\epsilon \sigma A)$ con unidades de $[K]^4[s]$. Aunque se hayan supuesto constantes, el valor del resto de parámetros de la ecuación son desconocidos.

Durante los ensayos, las temperaturas del clúster de Raspberry Pi se midieron en varios puntos de funcionamiento. Estas mediciones permitieron calcular la potencia disipada en cada una de las tarjetas utilizando la ecuación anterior. Los resultados obtenidos proporcionan una relación clara y directa entre la temperatura del clúster y la potencia disipada, facilitando la evaluación del comportamiento térmico del clúster bajo diferentes condiciones operativas.

5.3. Experimentos realizados para evaluación del consumo energético frente al tiempo de ejecución para distintas configuraciones del clúster.

La caracterización del sistema se ha llevado a cabo evaluando la influencia de cada uno de los grados de libertad del sistema sobre las variables observadas. Las variables empleadas para la caracterización del rendimiento del clúster han sido el tiempo de ejecución requerido por cada configuración para completar el benchmark y la evolución de la temperatura de la CPU durante la ejecución. Con estas dos variables se calcularán las métricas de speed up y eficiencia energética para comparar cuantitativamente unas configuraciones frente a otras.

La Figura 8 muestra con un diagrama de flujo las condiciones que se han seguido en cada experimento. Los cuatro primeros experimentos se centran en analizar el rendimiento de un único nodo. Debido a esto, cada experimento trata de aislar al máximo la relación entre las variables observadas con cada uno de los grados de libertad. Por el contrario, los experimentos que analizan el clúster al completo no se centran en obtener la relación de cada parámetro con la eficiencia del clúster. Estos experimentos dan cuenta de qué configuraciones son las más convenientes sin analizar al detalle la influencia de los parámetros escogidos durante la ejecución con los resultados. Los experimentos 5 y 6 pretenden servir como un repositorio que recoja las configuraciones óptimas independientemente de cuales sean los objetivos de uso del clúster.

5.3.1. Experimento 1. Influencia de la frecuencia de trabajo de la CPU sobre el consumo energético del chip y su capacidad de cómputo.

Existen herramientas que permiten modificar la frecuencia de trabajo del procesador de cada chip Raspberry Pi. Modificando la frecuencia de trabajo del procesador de una tarjeta se puede controlar la potencia de cómputo que puede alcanzar, así como su consumo energético. Si se somete a un cierto estrés controlado al procesador de un nodo de cómputo cuando trabaja a distintas frecuencias se puede evaluar la relación del consumo energético frente a la potencia de cómputo que tiene.

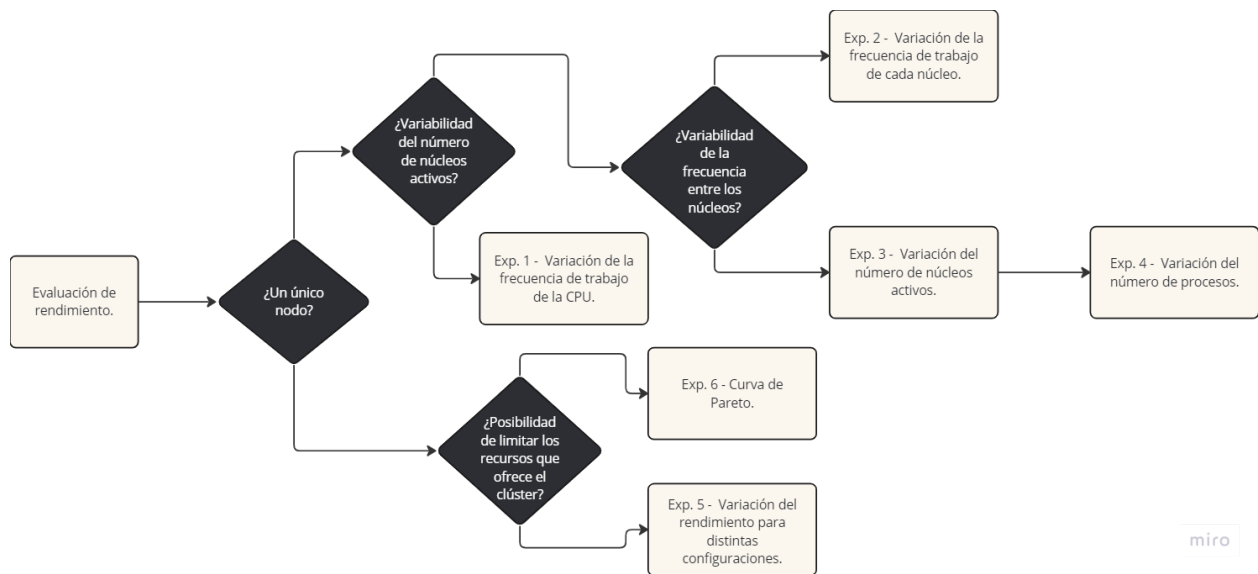


Figura 8: Diagrama de flujo de los experimentos de caracterización del clúster.

La frecuencia de trabajo de la CPU de cada nodo del clúster se puede modificar a partir de herramientas software gratuitas como `CPUfrequtils` [24]. La siguiente consola muestra los comandos empleados para modificar la frecuencia de trabajo de todos núcleos de un mismo chip. Esta herramienta incluye además un comando para conocer la frecuencia a la que está trabajando cada uno de los núcleos (o cores).

CPUfrequtils.

```

# Instalacion
pi@nodo0:~ $ sudo apt update
pi@nodo0:~ $ sudo apt install CPUfrequtils
# Verificar las frecuencias disponibles en cada core.
pi@nodo0:~ $ CPUfreq-info | grep current
# Modificar la frecuencia.
pi@nodo0:~ $ sudo CPUfreq-set -f <frecuencia en kHz>

```

Este experimento no ha llevado a cabo paralelización en varios procesos MPI y ha sido sustentado en únicamente un núcleo de un solo nodo del clúster. Esta configuración pretende esclarecer lo máximo posible la dependencia de las variables observadas con la frecuencia de trabajo del chip que soportará la carga computacional de la simulación, aislando lo máximo posible la influencia del resto de variables controlables sobre el tiempo de ejecución.

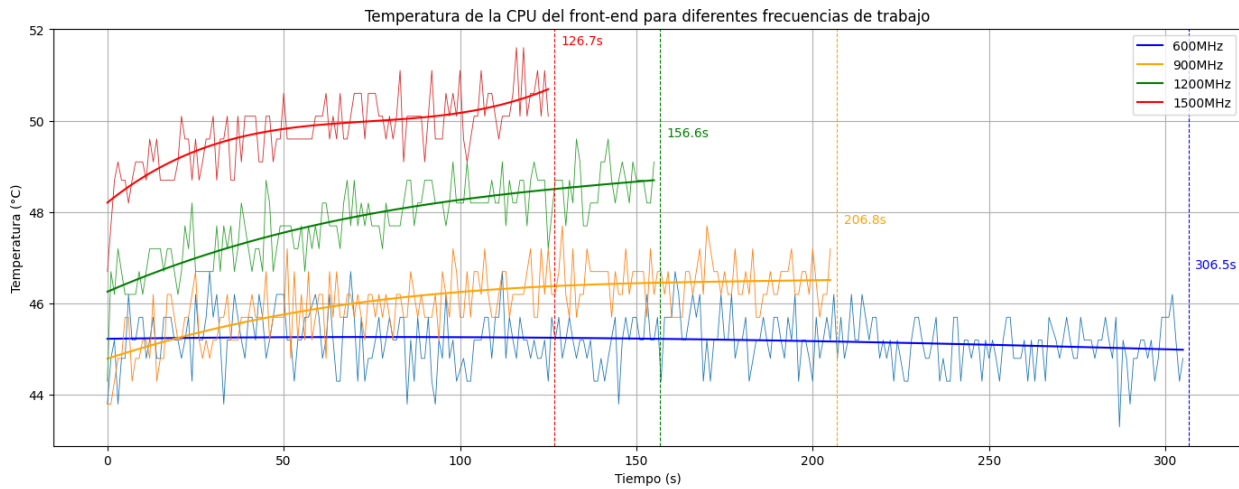


Figura 9: Evolución de la temperatura de la CPU con el tiempo para distintas frecuencias de trabajo del procesador. Experimento realizado para el *front-end* sometiénolo a una carga computacional controlada 5.1. Los tiempos de ejecución para cada configuración vienen recogidos siguiendo el código de colores de la leyenda.

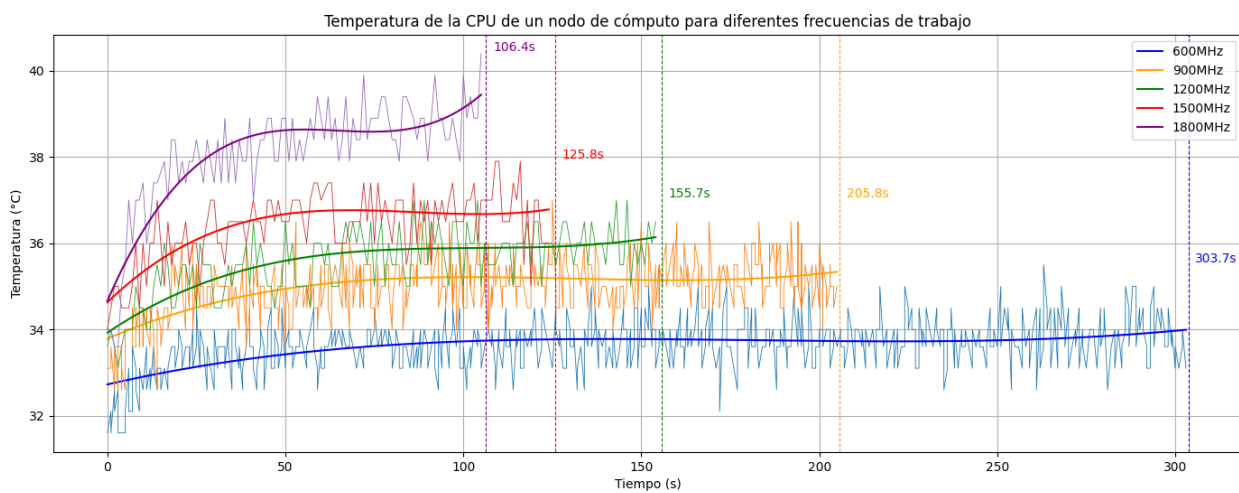


Figura 10: Evolución de la temperatura de la CPU con el tiempo para distintas frecuencias de trabajo del procesador cuando se somete a una carga computacional controlada 5.1. Experimento realizado para el *nodo1*. Los tiempos de ejecución para cada configuración vienen recogidos siguiendo el código de colores de la leyenda.

La Figura 9 recoge la evolución de la potencia consumida por la CPU, parametrizada por su temperatura, cuando se somete a una carga computacional exigente. El consumo energético se eleva sustancialmente durante la ejecución. A medida que aumenta la frecuencia del procesador, las prestaciones del chip aumentan y este efecto se hace más notable. En contraposición, los tiempos de ejecución se reducen con el aumento de la frecuencia. De la Figura 9 puede deducirse que, al menos dentro del rango 600 MHz a 1,5 GHz, el tiempo de ejecución sigue una dependencia de $1/\text{frecuencia}$ del chip.

Los nodos de cómputo, al carecer de interfaz gráfica alcanzan anchos de banda de hasta 1,8 GHz, 300 MHz más de los que es capaz de ofrecer el nodo que opera como *front-end*. Además, deben soportar menor carga computacional que él al no cargar con tantas acciones en segundo plano externas a la simulación. De ahí que las temperaturas de la CPU medidas para un estado de reposo, referidas a los valores iniciales mostrados en las Figuras 9 y 10, son de entorno a un 25 % más altas en el nodo de cómputo que opera como *front-end*.

Frecuencia	Tiempo de ejecución	Speed Up	Ahorro energético (%)
600 MHz	303,66	1,0	71,05 %
900 MHz	205,765	1,476	54,93 %
1200 MHz	155,704	1,95	53,62 %
1500 MHz	125,796	2,414	44,23 %
1800 MHz	106,362	2,855	0,0 %

Tabla 4: Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en un nodo de cómputo durante el experimento 1. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 10.

Nuevamente, como muestra la Figura 10 y la Tabla 4, la tendencia que siguen los tiempos de ejecución con la frecuencia de trabajo del chip cuando el programa es ejecutado solo por un nodo sigue siendo de aproximadamente $1/\text{frecuencia}$ del chip.

5.3.2. Experimento 2. Influencia de la frecuencia de trabajo de la CPU sobre el consumo energético del chip y su capacidad de cómputo para múltiples núcleos.

La herramienta CPUfrequtils 5.3.1, no está habilitada para modificar la frecuencia de trabajo a nivel de núcleo dentro de un chip Raspberry Pi. A pesar de todo, un experimento que emule de cierta manera esta situación tiene mucho interés. Este experimento pretende dar una aproximación de la influencia que tendría modificar la frecuencia de trabajo de cada uno de los núcleos de un

chip por separado, simulado que estuviese formado por tres núcleos independientes. Para esto cada nodo funcionará como un núcleo, emulando que los tres núcleos se encuentran en un mismo procesador.

Este experimento se ha llevado a cabo paralelizando la ejecución en tres procesos MPI, uno por cada nodo (que emula a un núcleo del procesador). La ejecución se ha sustentado por lo tanto en un núcleo de cada nodo de cómputo del clúster. De esta manera, cada tarjeta emula el comportamiento de un núcleo dentro de un chip común que engloba a todo el clúster. Esta configuración pretende esclarecer lo máximo posible la dependencia de las variables observadas con la frecuencia de trabajo de cada uno de los núcleos que soportarán la carga computacional de la simulación, aislando lo máximo posible la influencia del resto de variables controlables sobre el tiempo de ejecución.

El tiempo estimado para un proceso, que figura en la imagen, es un parámetro que devuelve el

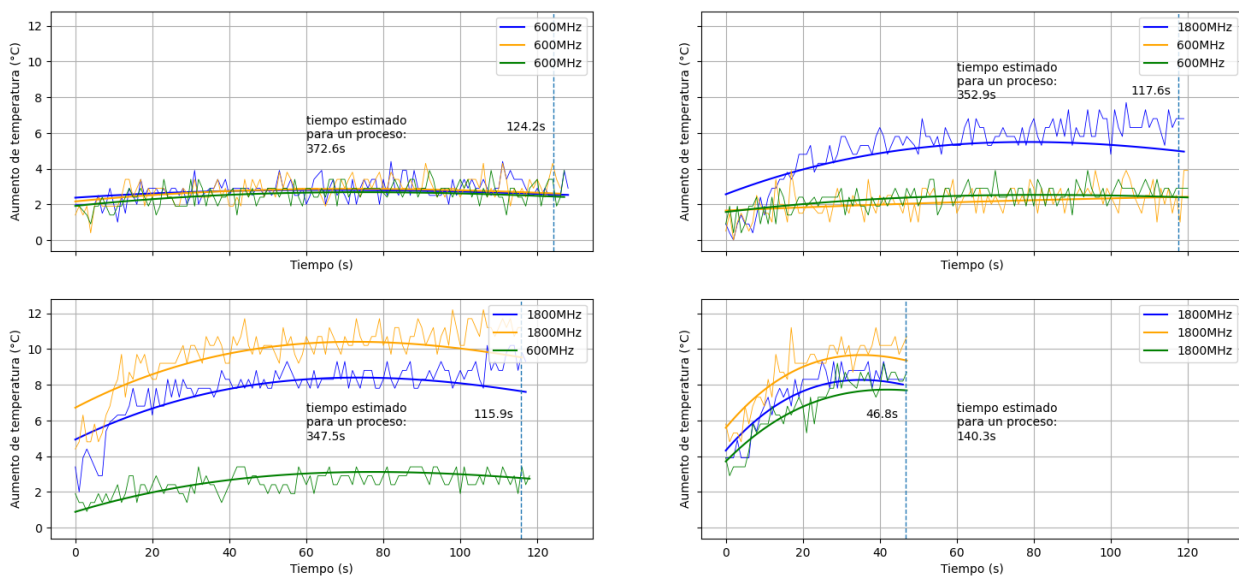


Figura 11: Evolución de la temperatura con el tiempo de la CPU de tres chips, emulando ser núcleos con control independiente dentro de un mismo procesador. Se estudia la influencia de la frecuencia de trabajo de cada núcleo sobre los tiempos de ejecución cuando se somete al procesador a una carga computacional controlada 5.1. La frecuencia de trabajo de cada “núcleo” se recoge en la leyenda siguiendo el código de colores de cada curva.

benchmark tras la ejecución. Es un indicador de la potencia de cómputo, estimada por el programa, del proceso más lento durante la ejecución. Refleja cuanto hubiera tardado ese proceso en terminar si hubiera procesado toda la información, estimando su capacidad de cómputo durante el ensayo. Las condiciones del ensayo dependerán de los parámetros de ejecución de cada configuración. Este indicador permite comparar como se reducen las prestaciones de cada proceso cuando se paraleliza la carga en varios de ellos.

Configuración	Tiempo de ejecución	Speed Up	Ahorro energético normalizado
600-600-600 MHz	124,205	1,0	41,99 %
1800-600-600 MHz	117,641	1,056	34,57 %
1800-1800-600 MHz	115,855	1,072	0,0 %
1800-1800-1800 MHz	46,779	2,655	31,5 %

Tabla 5: Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en los tres nodos de cómputo durante el experimento 2. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 11.

En las cuatro simulaciones que recoge la Figura 11, los chips que emulan ser los núcleos con mayor capacidad de cómputo (frecuencia de trabajo más alta) son los que más se calientan. El protocolo MPI no tiene información acerca de la capacidad de cómputo de cada chip. Por esto, diversificará el programa paralelizándolo en tres procesos equivalentes aunque alguno de los núcleos pueda procesar con mayor velocidad la información. La reducción en el tiempo de ejecución no será notablemente efectiva hasta que todos los núcleos trabajen a altas frecuencias. Al diversificar los recursos de forma equitativa, aquellos chips con menor capacidad de cómputo irán rezagados durante la ejecución, arrastrando con ellos a todo el sistema. El tiempo de ejecución, por lo tanto, estará limitado por el núcleo que presente las peores prestaciones.

Como se puede observar en la Tabla 5, un chip formado por núcleos con notables diferencias en su capacidad de cómputo implicará un desaprovechamiento de los recursos que son capaces de ofrecer los núcleos más potentes, que se verán limitados por aquellos que no lo son tanto. Lejos de suponer un ahorro energético, la ineficiencia en el reparto de los recursos en este tipo de configuraciones supone un consumo energético adicional que no se ve reflejado en términos de speed up durante la ejecución. En definitiva, tanto desde el prisma de la eficiencia energética como del de la capacidad de cómputo, un procesador eficiente siempre deberá tender a minimizar las diferencias entre las prestaciones de todos sus núcleos o a realizar un reparto de la carga computacional teniendo en cuenta la capacidad de los recursos subyacentes.

5.3.3. Experimento 3. Influencia del número de núcleos de la CPU activos sobre el consumo energético del chip y su capacidad de cómputo.

La ejecución de cualquier script facilitado por GROMMACS permite su ejecución con soporte de OpenMP. OpenMP es un software habilitado para controlar la cantidad de hilos dentro de un procesador que estarán implicados en un proceso de ejecución. Este software solo es capaz de controlar como se reparten los recursos dentro de cada nodo. Por lo tanto, en computación de altas

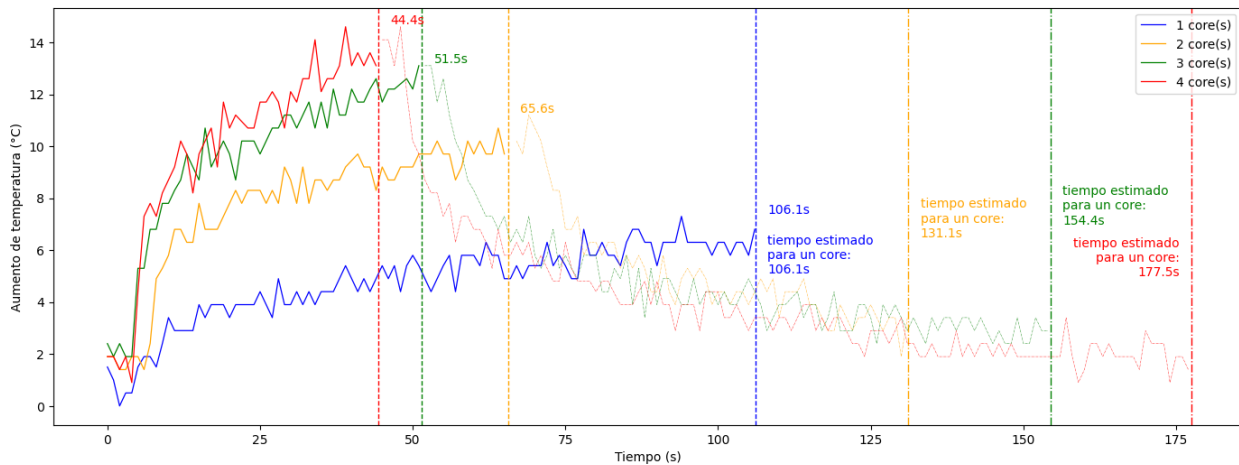


Figura 12: Evolución de la temperatura con el tiempo de la CPU de un nodo de cómputo cuando su procesador está sometido a una carga computacional controlada 5.1. Simulación llevada a cabo variando el número de threads OpenMP en los que se paraleliza la simulación. El número de núcleos del procesador (o threads) implicados se recoge en la leyenda siguiendo su código de colores.

prestaciones, el parámetro a controlar por OpenMP será el número de threads implicados en cada proceso MPI (que se asociará al número de nodos). Como muestra la siguiente consola, el parámetro `-ntomp` determina cuantos núcleos del procesador van a destinarse a cargar con simulación de cada proceso MPI.

Este experimento ha paralelizado la ejecución en un rango de 1-4 threads habilitados de la CPU de uno de los nodos de cómputo del clúster. Todos los threads habilitados durante la simulación trabajan a una frecuencia estable de 1,8 GHz. La ejecución se ha dividido en cada simulación en los threads homónimos al número de núcleos (o cores) que se busca destinar a la ejecución, asegurando así su paralelización. Durante todas las simulaciones se ha fijado el número de procesos MPI a uno, indicando que la simulación se alojara en un único nodo. Esta configuración pretende esclarecer lo máximo posible la dependencia de las variables observadas con el número de núcleos que soportarán la carga computacional de la simulación, aislando lo máximo posible la influencia del resto de variables controlables sobre el tiempo de ejecución.

Comparando la Figura 12 con la última gráfica de la Figura 11 se puede cuantificar como de fiel a la realidad es la simulación que se hizo en el experimento anterior. Teóricamente, si el aumento de la capacidad de cómputo fuera lineal con el número de procesos, emplear tres núcleos de un mismo procesador debería tener las mismas prestaciones que emplear tres núcleos relativos a tres nodos de cómputo distintos (como en el Experimento 2). Uno de los objetivos del experimento anterior era comprobar hasta que punto las consideraciones que se hicieron eran ciertas. La Tabla

Configuración	Tiempo de ejecución	Speed Up	Ahorro energético normalizado
1 core(s)	106,13	1,0	5,71 %
2 core(s)	65,577	1,618	0,0 %
3 core(s)	51,496	2,061	16,71 %
4 core(s)	44,391	2,391	28,24 %

Tabla 6: Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en los tres nodos de cómputo durante el experimento 3. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 12.

7 muestra como los núcleos “*independientes*” del experimento 2 están hasta un 6,92 % por encima de la eficiencia real de la CPU de una placa al comparar los tiempos de ejecución con el peor caso de ambos experimentos. En el tercer experimento, a las pérdidas inherentes derivadas de la paralelización se le suman las debidas a la carga que supone para el procesador repartir el trabajo entre sus distintos núcleos. Esto último se observa en que los tiempos de ejecución estimados de la Figura 12, que dan cuenta de la eficiencia computacional de cada núcleo durante la ejecución, aumentan con el número de cores implicados en el proceso de ejecución.

Aunque la eficiencia no sea perfecta, resulta muy beneficioso emplear toda la capacidad que ofrece el procesador. La Tabla 6 muestra como al utilizar los cuatro núcleos de los que dispone a frecuencia máxima (1,8 GHz) los tiempos de ejecución se reducen considerablemente. Además de aumentar las prestaciones, dedicar todos los núcleos disponibles a la ejecución del benchmark supone un ahorro energético con respecto al resto de configuraciones. Aunque parezca sorprendente, emplear todos los núcleos a la ejecución limita considerablemente la actividad en segundo plano haciendo que, en cómputo general, el consumo energético termine reduciéndose. La considerable reducción en los tiempos de ejecución al aumentar los recursos dedicados es un buen indicador de que el clúster funciona como cabría esperar.

5.3.4. Experimento 4. Influencia del número de procesos MPI en los que se diversifique la carga sobre el consumo energético del chip y su capacidad de cómputo. Experimento realizado para un único nodo.

En el experimento anterior se observó como, al aumentar la cantidad de núcleos efectivos de un procesador, las prestaciones de la CPU aumentaban. Esto es gracias a que OpenMP, internamente, separó las tareas en tantos hilos paralelizables como núcleos activos de los que disponía el clúster. Este experimento trata de enfocar el problema desde otro ángulo. En él, se ha paralelizado la ejecución en un rango de 1-4 procesos generados por MPI. Para asegurar la paralelización,

Muestra	Tiempo de ejecución	Tiempo de ejecución suponiendo un único núcleo	Speed up porcentual
Exp. 2	46,8 s	140,3 s	127 %
Exp. 3	51,5 s	154,4 s	107 %
Exp. 4	54,3 s	162,9 s	96 %

Tabla 7: Comparación de los tiempos de ejecución y eficiencias relativas para los procesadores emulados en el experimento 2 (núcleos pertenecientes a distintos procesadores) con los núcleos de un solo procesador empleados en el experimento 3 y con el experimento 4, en el que se emplean procesos en lugar de hilos.

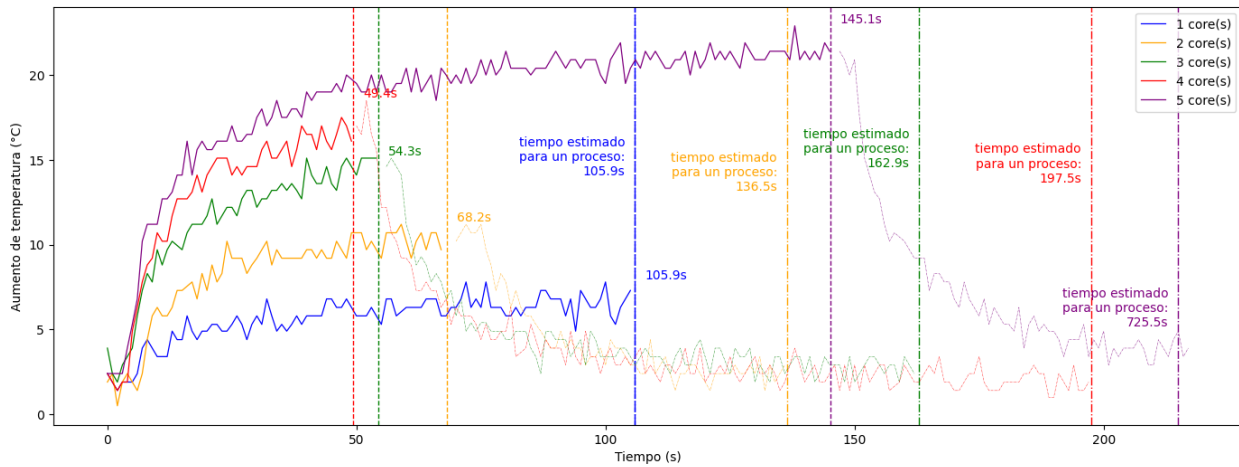


Figura 13: Evolución de la temperatura con el tiempo de la CPU de un nodo de cómputo cuando se somete al procesador a una carga computacional controlada 5.1. Simulación llevada a cabo variando el número de procesos en los que se paraleliza la simulación. El número de procesos implicados se recoge en la leyenda siguiendo el código de colores de las curvas.

internamente cada proceso se asociará a un thread (creado por OpenMP) que se ejecutará en un núcleo distinto. Todos los núcleos habilitados durante la simulación trabajan a una frecuencia estable de 1,8 GHz. Esta configuración pretende esclarecer lo máximo posible la dependencia de las variables observadas con el número de procesos en que se puede dividir la carga computacional de la simulación, aislando lo máximo posible la influencia del resto de variables controlables sobre el tiempo de ejecución.

Al aumentar el número de procesos, el ratio tiempo invertido equivalente a un proceso aumenta. Este fenómeno, hasta 4 procesos, ya se podría haber intuido observando el comportamiento del sistema al emplear múltiples núcleos. En este caso, a las pérdidas de eficiencia inherentes de cada núcleo al estar utilizando múltiples cores dentro de un mismo procesador, se suman las pérdidas que pueda ocasionar el protocolo MPI al dividir el problema en procesos que puede que no estén

Configuración	Tiempo de ejecución	Speed Up	Ahorro energético normalizado
1 proceso(s)	105,898	1,37	75,93 %
2 proceso(s)	68,242	2,126	59,73 %
3 proceso(s)	54,322	2,671	33,43 %
4 proceso(s)	49,391	2,938	0,0 %
5 proceso(s)	145,104	1,0	-691,66 %

Tabla 8: Comparación de los tiempos de ejecución y eficiencias energéticas relativas para las distintas configuraciones probadas en los tres nodos de cómputo durante el experimento 4. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 13.

siempre totalmente optimizados (Ver Tabla 7). Debido a esto, el speed up de utilizar más procesos, lejos de ser lineal, es menor que el que se obtuvo en el experimento anterior. Esto se debe a que el modelo de programación paralela MPI trabaja mediante un protocolo de mensajería que es capaz de comunicar nodos del clúster que no comparten espacio de memoria. Emplear este protocolo dentro de un mismo procesador genera un consumo de recursos innecesarios que se evita al trabajar con hilos. La eficiencia se reduce ligeramente al utilizar un núcleo dedicado a cada proceso en lugar de aunar la potencia de los cuatro núcleos en un único proceso.

Aún así, dentro del rango de 1 a 4 procesos el comportamiento es muy similar al que se observaba en el Experimento 3. Por el contrario, cuando se divide el problema en más de cuatro procesos (curva morada, Figura 13) el procesador no tiene suficientes núcleos como para emplear cada uno de ellos en un solo proceso. Ocurre algo similar a lo que se observa en la figura del experimento 2, donde unos núcleos sostenían consumos elevados durante tiempos excesivamente altos debido a que la ejecución no terminaba hasta que los núcleos más rezagados habían finalizado sus tareas. La curva asociada a cinco procesos es la que más consumo genera y además la que emplea mayor tiempo en completar la ejecución. Esto se debe a que uno de los núcleos de la CPU debe ejecutar dos procesos, aumentando considerablemente el tiempo de ejecución. El ahorro energético de cada configuración que se muestra en la Tabla 8 está normalizado al caso en el que se emplean cuatro procesos debido a este motivo. Utilizar el caso de cinco procesos como referencia no sería ilustrativo como referencia de consumo energético debido a la cantidad de recursos que se desperdician en esa configuración.

5.3.5. Experimento 5. Estudio de la capacidad de cómputo y consumo energético del clúster al utilizar distintas configuraciones cuando todos los nodos de cómputo están activos.

Para poder comparar la efectividad de cada configuración se ha paralelizado el proceso de ejecución en todos los núcleos de los que dispone el clúster. Variando únicamente el número de procesos MPI y el número de núcleos dedicados a cada proceso manteniendo el producto de ambas constante. El resto de variables controlables se han mantenido constantes durante el experimento a fin de aislar su influencia sobre la eficiencia del sistema. En todas las simulaciones los doce núcleos del clúster (cuatro núcleos por nodo) han trabajado a 0,6 GHz. La Figura 14 pretende mostrar cómo afectan al consumo energético y a la eficiencia computacional del clúster a cada una de las configuraciones posibles cuando el clúster utiliza todos los recursos de los que dispone. Las simulaciones se han llevado a cabo en todos los nodos de cómputo bajo las mismas condiciones, dando lugar a gráficas que no presentan diferencias reseñables entre sí.

Puede verse como al emplear un núcleo aislado para cada proceso, el consumo del clúster aumenta sustancialmente respecto al resto de configuraciones. Por el contrario, la configuración que reduce el número de procesos a la cantidad de nodos del clúster presenta el mayor ahorro energético. Todos estos datos se recogen en la Tabla 9. Sobrecargar al sistema con la comunicación asociada a la mensajería MPI, genera un consumo energético adicional que desaparece cuando los núcleos dentro de un mismo procesador trabajan solamente en un proceso.

Configuración	Tiempo de ejecución	Speed Up	Ahorro energético normalizado
3 Procesos	49,6 s	4,30	17,37 %
4 Procesos	49,3 s	4,32	15,90 %
6 Procesos	52,0 s	4,09	10,28 %
12 Procesos	47,2 s	4,51	0 %

Tabla 9: Comparación de los tiempos de ejecución y eficiencias relativas para las distintas configuraciones probadas durante el experimento 5. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 14.

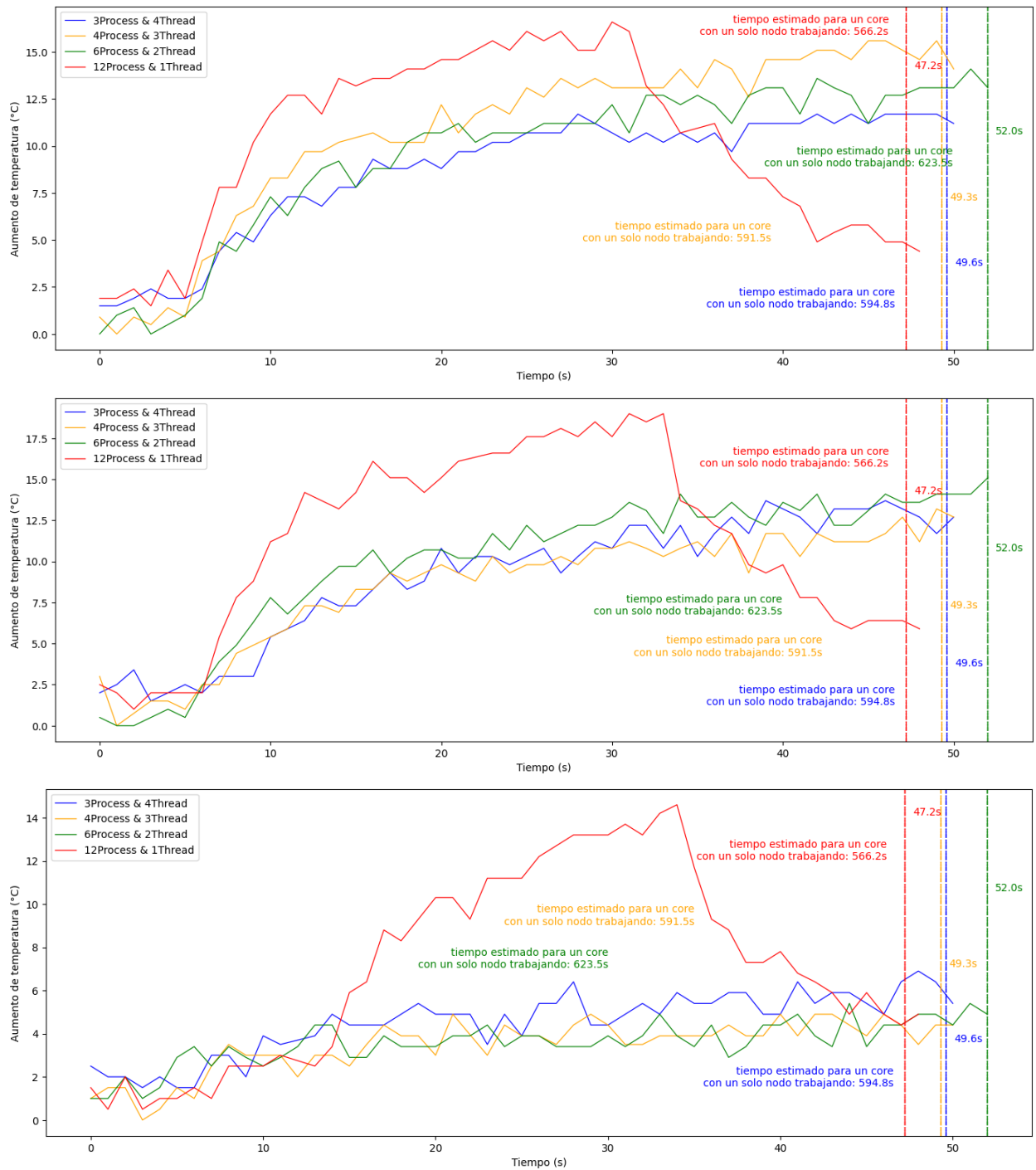


Figura 14: Evolución de la temperatura con el tiempo de la CPU de los tres nodos de cómputo cuando se somete al procesador a una carga computacional controlada 5.1. Se han llevado a cabo simulaciones variando el número de procesos MPI manteniendo en todas las simulaciones el número de hilos totales constante e igual al número de núcleos de los que disponibles en el clúster (12 cores).

5.3.6. Experimento 6. Optimización de la capacidad de cómputo con el consumo energético del clúster para distintas configuraciones cuando todos los nodos de cómputo están activos.

Para caracterizar completamente el clúster es necesario disponer de una muestra de simulaciones relativamente alta que comparta las suficientes características como para que se puedan observar resultados ilustrativos acerca de su influencia sobre la efectividad del clúster en la ejecución de software de altas prestaciones. En este experimento se han realizado simulaciones variando la frecuencia de trabajo de los procesadores de cada nodo de cómputo, el número de procesos MPI en los que se paraleliza la simulación y el número de threads OpenMP implicados en cada proceso.

Durante este ensayo no se ha pretendido aislar cada una de las variables para observar su influencia sobre el consumo energético y la potencia de cómputo. Por el contrario, sin importar la influencia de cada variable sobre la eficiencia del clúster, se ha experimentado con todas las configuraciones disponibles del sistema para obtener cuales son las mejores configuraciones dentro del espacio de soluciones del sistema. La Tabla 10 recoge todo el espacio de configuraciones que se ha puesto a prueba durante el ensayo. Todas las configuraciones han sido construidas cumpliendo los siguientes criterios:

1. El número de procesos MPI debían ser un múltiplo del número de nodos para asegurar que todos los procesadores tuvieran la misma participación sobre la ejecución.
2. La frecuencia de trabajo de todos los nodos de cómputo debía ser la misma y entrar dentro del rango 0,6-1,8 GHz
3. El producto procesos MPI x núcleos implicados en cada proceso debía ser menor a 12 (la cantidad de núcleos disponibles).

La Figura 15 muestra el espacio de soluciones del sistema como un mapa de puntos donde cada solución está caracterizada por los parámetros de tiempo de ejecución y consumo energético. El consumo total del sistema en toda la ejecución se ha calculado como la integral en el tiempo de la diferencia entre la temperatura a la cuarta potencia de la CPU de los tres nodos de cómputo y la temperatura ambiente a la cuarta potencia. A diferencia del resto de experimentos en este se ha optado por este criterio independientemente del número de nodos de cómputo que durante ese tiempo destinaran sus recursos a la ejecución. Se ha empleado este criterio debido a que durante todo el ensayo el clúster no ha empleado herramientas de gestión de recursos, de modo que todos los recursos que no se hayan empleado en la ejecución del software no se podrían haber aprovechado por otros procesos.

5.3.7. Curva de Pareto

Se han analizado los resultados obtenidos del ensayo del clúster con el objetivo de representar la curva de Pareto del espacio de soluciones. Esta curva se ha empleado para identificar las configuraciones óptimas tanto en términos de maximización de la capacidad de cómputo como de optimización del consumo energético.

La curva de Pareto es una herramienta fundamental en la toma de decisiones multicriterio, ya que permite visualizar las soluciones que ofrecen el mejor compromiso entre dos objetivos opuestos. En este caso, los dos objetivos considerados son:

- Maximizar la capacidad de cómputo del clúster de Raspberry Pi.
- Minimizar el consumo energético asociado al funcionamiento del clúster.

Para construir la curva de Pareto, se han seguido los siguientes pasos:

1. Se han medido y registrado las temperaturas y las correspondientes potencias disipadas en diferentes configuraciones del clúster de Raspberry Pi.
2. Se ha calculado la capacidad de cómputo de cada configuración, considerando el tiempo de ejecución que necesitaba el sistema para resolver un problema cuya capacidad de cómputo es conocida.
3. Se ha evaluado el consumo energético de cada configuración utilizando la potencia disipada.
4. Se ha graficado el espacio de soluciones, donde cada punto representa una configuración específica del clúster de Raspberry Pi, con la capacidad de cómputo en el eje x y el consumo energético en el eje y .

En la representación gráfica, la curva de Pareto se define como el conjunto de puntos que no son dominados por ningún otro punto en el espacio de soluciones. Un punto A domina a un punto B si A tiene una mayor capacidad de cómputo y un menor consumo energético que B . Los puntos en la curva de Pareto representan las soluciones más eficientes, ofreciendo el mejor compromiso entre los dos objetivos.

Como se puede observar en la Figura 15, la curva de Pareto delimita las configuraciones óptimas. Las configuraciones a la derecha de la curva no son óptimas, ya que existe al menos una configuración en la curva que ofrece una mejor capacidad de cómputo con un consumo energético igual o menor. De manera similar, las configuraciones por encima de la curva no son óptimas, ya que existe al menos una configuración en la curva que ofrece un menor consumo energético con una capacidad de cómputo igual o mayor.

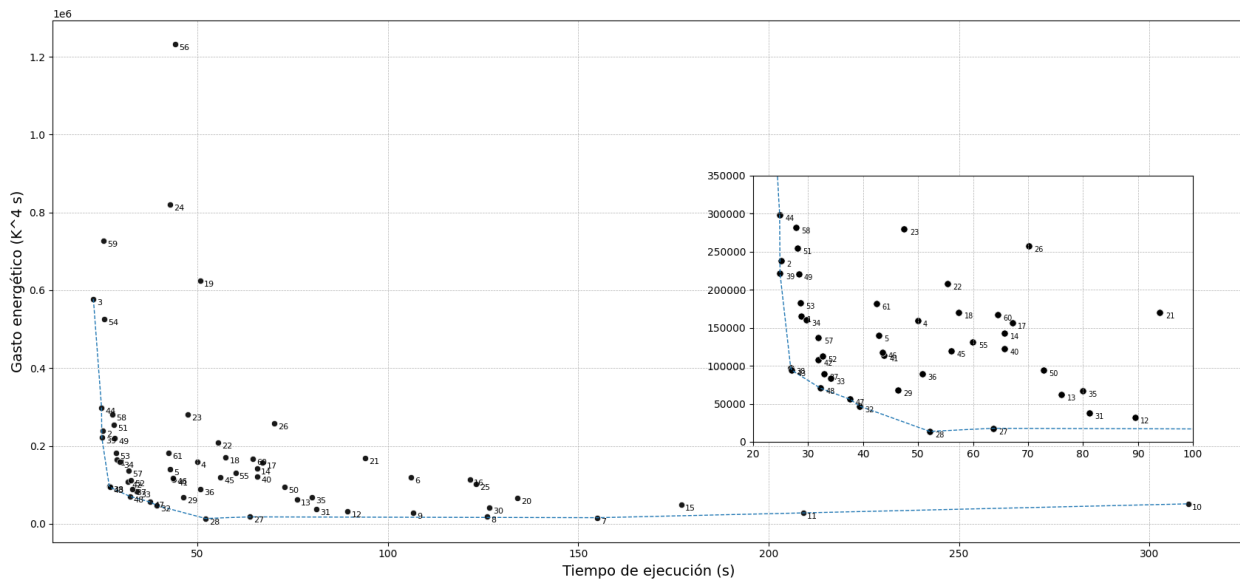


Figura 15: Tiempo de ejecución y consumo energético del espacio de configuraciones del clúster cuando se somete al procesador a una carga computacional controlada 5.1. La curva de Pareto recorre las configuraciones que minimizan el consumo energético para cada tiempo de ejecución.

La representación de la curva de Pareto ha permitido identificar las configuraciones óptimas del clúster de Raspberry Pi, ya sea para maximizar la capacidad de cómputo o para optimizar el consumo energético. Esta información es crucial para la toma de decisiones informadas, permitiendo seleccionar la configuración que mejor se adapte a las necesidades específicas del usuario.

Conf.	Procesos MPI	Threads	Frecuencia de trabajo	Speed up	Ahorro energético normalizado	Speed up /(1-ahorro)
1	12	1	1200MHz	10,779	58,97 %	26,26
2	12	1	1500MHz	12,386	49,22 %	24,39
3	12	1	1800MHz	13,681	31,65 %	20,02
4	12	1	600MHz	6,214	63,87 %	17,19
5	12	1	900MHz	7,247	58,87 %	17,62
6	1	12	600MHz	2,926	54,8 %	6,47
7	1	1	1200MHz	2,001	20,84 %	2,53
8	1	1	1500MHz	2,46	7,04 %	2,65
9	1	1	1800MHz	2,908	3,84 %	3,02
10	1	1	600MHz	1,0	0,0 %	1,00

Conf.	Procesos MPI	Threads	Frecuencia de trabajo	Speed up	Ahorro energético normalizado	Speed up /(1-ahorro)
11	1	1	900MHz	1,484	5,26 %	1,57
12	1	2	1200MHz	3,468	45,98 %	6,42
13	1	2	1500MHz	4,077	31,02 %	5,91
14	1	2	1800MHz	4,721	18,54 %	5,79
15	1	2	600MHz	1,752	36,88 %	2,77
16	1	2	900MHz	2,552	20,33 %	3,20
17	1	3	1200MHz	4,62	40,82 %	7,81
18	1	3	1500MHz	5,402	39,19 %	8,88
19	1	3	1800MHz	6,113	9,55 %	6,76
20	1	3	600MHz	2,316	53,32 %	4,96
21	1	3	900MHz	3,301	35,0 %	5,08
22	1	4	1200MHz	5,605	52,77 %	11,87
23	1	4	1500MHz	6,535	42,15 %	11,29
24	1	4	1800MHz	7,237	26,06 %	9,79
25	1	4	600MHz	2,519	51,63 %	5,21
26	1	4	900MHz	4,424	44,08 %	7,91
27	3	1	1200MHz	4,87	53,48 %	10,44
28	3	1	1500MHz	5,951	53,4 %	12,75
29	3	1	1800MHz	6,698	22,62 %	8,66
30	3	1	600MHz	2,451	46,21 %	4,56
31	3	1	900MHz	3,823	42,84 %	6,68
32	3	2	1200MHz	7,877	59,28 %	19,27
33	3	2	1500MHz	9,088	48,0 %	17,47
34	3	2	1800MHz	10,462	36,24 %	16,35
35	3	2	600MHz	3,879	57,43 %	9,09
36	3	2	900MHz	6,106	51,32 %	12,54
37	3	3	1200MHz	9,447	58,45 %	22,78
38	3	3	1500MHz	11,565	57,44 %	27,11

Conf.	Procesos MPI	Threads	Frecuencia de trabajo	Speed up	Ahorro energético normalizado	Speed up / (1-ahorro)
39	3	3	1800MHz	12,462	41,91 %	21,43
40	3	3	600MHz	4,724	56,86 %	10,98
41	3	3	900MHz	7,086	55,21 %	15,81
42	3	4	1200MHz	9,751	58,19 %	23,39
43	3	4	1500MHz	11,488	58,26 %	27,54
44	3	4	1800MHz	12,501	36,38 %	19,63
45	3	4	600MHz	5,535	62,63 %	14,78
46	3	4	900MHz	7,125	54,74 %	15,75
47	6	1	1200MHz	8,247	57,71 %	19,43
48	6	1	1500MHz	9,622	52,89 %	20,37
49	6	1	1800MHz	10,975	35,11 %	16,88
50	6	1	600MHz	4,258	57,05 %	9,91
51	6	1	900MHz	11,07	66,22 %	32,81
52	6	2	1200MHz	9,523	56,72 %	22,06
53	6	2	1500MHz	10,851	46,36 %	20,22
54	6	2	1800MHz	12,111	23,63 %	15,85
55	6	2	600MHz	5,172	60,3 %	13,03
56	6	2	900MHz	7,03	32,82 %	10,49
57	9	1	1200MHz	9,73	55,84 %	22,02
58	9	1	1500MHz	11,178	41,22 %	18,98
59	9	1	1800MHz	12,242	17,51 %	14,83
60	9	1	600MHz	4,813	54,43 %	10,55
61	9	1	900MHz	7,31	50,61 %	14,80

Tabla 10: Información de todas las configuraciones empleadas durante el experimento 6. Esta tabla se ha obtenido de los valores medios en los tiempos de ejecución, temperatura de la CPU de los tres nodos recogidos en la Figura 15.

6. Conclusiones y trabajo futuro.

El principal objetivo de este trabajo fue desarrollar un clúster de bajo coste y consumo que sirviera como una prueba de concepto incorporando todas las herramientas necesarias para operar en un entorno de computación de altas prestaciones (HPC).

A partir de este objetivo, surgieron rutas de trabajo secundarias, como la obtención de la curva de Pareto del sistema (objetivo 4). Gracias a esta métrica, el sistema quedó totalmente caracterizado, permitiendo a los usuarios identificar visualmente la configuración óptima según los requisitos de su aplicación. La gráfica de Pareto permite visualizar para cualquier aplicación aquellas configuraciones con mejor compromiso entre la capacidad de cómputo del sistema y su consumo energético.

Se han cumplido todos los objetivos propuestos para el trabajo. Los resultados obtenidos del análisis de los costes asociados a un clúster construido a partir de placas Raspberry Pi 4 (objetivo 1) lo posicionan como una opción viable y accesible para proyectos que busquen una buena relación entre la capacidad de cómputo y el consumo eléctrico e inversión económica asociados. Se han aplicado modelos de programación paralela (objetivos 2, 3) para optimizar los recursos hardware que ofrecía el clúster con una serie de experimentos que han servido a su vez para caracterizar sus posibles configuraciones de funcionamiento.

El proceso de configuración no fue fácil. A pesar de la extensa documentación existente sobre computación de altas prestaciones, la configuración de un clúster como este requiere un conocimiento pleno de todos los pasos precedentes que se han ido tomando durante el proceso. Se trata de un trabajo muy minucioso, en el que todo debe encajar. Durante el proceso surgieron inconvenientes e incompatibilidades, generalmente derivados de que los procesadores ARM que montan las placas Raspberry Pi no están desarrollados para la computación de altas prestaciones. La mayoría de software científico que se buscó terminó siendo incompatible con la arquitectura ARM.

No obstante, estos inconvenientes han fortalecido el aprendizaje y la integración de conceptos propios de la ejecución de software científico en HPC como la concurrencia, los procesos o los hilos, tan nombrados en este documento. Al evaluar el rendimiento del clúster en los distintos experimentos, se observó como la máquina fue capaz de cumplir con todas las expectativas que existían sobre su escalabilidad. Los experimentos realizados respaldan la efectividad del sistema; mostrando mejoras significativas en el rendimiento del equipo gracias a la optimización de los recursos hardware mediante el uso de procesos e hilos controlados por las API MPI y OpenMP.

El estudio de la escalabilidad de un clúster construido por placas de este tipo (objetivo 5) tan solo se ha podido cumplir de forma parcial. Como trabajo futuro, hubiera sido interesante comparar el rendimiento del clúster con el de una arquitectura más compleja. Por ejemplo, un último experimento muy ilustrativo podría haber sido comparar los tiempos de ejecución de un ordenador al uso, con un procesador mucho más potente y sofisticado que permitiera cuantificar hasta qué punto se han podido llegar a optimizar los recursos hardware, mucho más limitados, con los que contaba el clúster. Otra línea de trabajo muy interesante que queda pendiente es la de escalar el clúster a un número significativamente mayor de placas. Con esta expansión se hubiera podido cuantificar la escalabilidad del sistema cuando la cantidad de nodos hábiles aumenta.

En conclusión, en este Trabajo Fin de Máster se ha logrado desarrollar una prueba de concepto de un clúster de HPC, integrando protocolos y técnicas esenciales para operar en un entorno de computación de altas prestaciones. A pesar de las dificultades encontradas, especialmente con la compatibilidad del software científico con la arquitectura ARM de las placas Raspberry Pi, el proyecto ha fortalecido mis conocimientos sobre la computación de altas prestaciones gracias a la integración de conceptos como los procesos e hilos en entornos concurrentes. Los resultados experimentales demostraron mejoras significativas en rendimiento y escalabilidad. Para futuros trabajos, sería valioso comparar el clúster con arquitecturas más complejas y escalar el sistema con un mayor número de nodos para evaluar su comportamiento en escenarios más amplios.

Referencias

- [1] BSC. Proyecto Montblanc. 2013. Accessed: 2024-08-11.
- [2] Rocío Carratalá, Sandra Catalán, Sergio Iserte, and Sergio López. *Construya su propio supercomputador con Raspberry Pi*. Editorial Desconocida, May 2021. Tapa blanda o bolsillo, 17x1x24 cm.
- [3] TOP500. TOP500. <https://www.top500.org>. Accessed: 2024-05-18.
- [4] Netlib. Linpack. <http://www.netlib.org/linpack/>. Accessed: 2024-05-18.
- [5] Netlib. High-Performance Benchmarking. <http://www.netlib.org/benchmark/hpl/>. Accessed: 2024-05-18.
- [6] Frontier - HPE Cray EX235a, AMD optimized 3rd generation EPYC 64c 2ghz, AMD Instinct MI250X, Slingshot-11, HPE. DOE/SC/Oak Ridge National Laboratory, United States, 2023. 8,699,904 cores, 1,206.00 petaflops, 1,714.81 theoretical petaflops, 22,786 kW.
- [7] Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52c 2.4ghz, Intel Data Center GPU Max, Slingshot-11, Intel. DOE/SC/Argonne National Laboratory, United States, 2023. 9,264,128 cores, 1,012.00 petaflops, 1,980.01 theoretical petaflops, 38,698 kW.
- [8] Eagle - Microsoft NDv5, Xeon Platinum 8480C 48c 2ghz, NVIDIA H100, NVIDIA Infini-band NDR, Microsoft Azure. Microsoft Azure, United States, 2023. Accessed: 2024-08-13.
- [9] Green500. Green500. <http://www.green500.org/>. Accessed: 2024-05-18.
- [10] Antônio José Alves Neto, José Aprígio Carneiro Neto, Edward David Moreno. The development of a low-cost big data cluster using Apache Hadoop and Raspberry Pi. A complete guide, Computers and Electrical Engineering. *Volume 104, Part A, 2022, 108403*,. Accessed: 2024-08-15.
- [11] Alberto Corpas, Luis Costero, Guillermo Botella, Francisco D. Igual, Carlos García, and Manuel Rodríguez. Acceleration and energy consumption optimization in cascading classifiers for face detection on low-cost arm big. little asymmetric architectures. *International Journal of Circuit Theory and Applications*, 46(9):1756–1776, September 2018.
- [12] OpenMP. OpenMP. <https://www.openmp.org>. Accessed: 2024-05-28.
- [13] MPI. MPI. <https://www.mpi-forum.org>. Accessed: 2024-05-28.

- [14] Mario Romero, Hanna Hasselqvist, and Gert Svensson. Supercomputers keeping people warm in the winter. *ICT for Sustainability 2014, ICT4S 2014*, pages 324–332, 01 2014. Accessed: 2024-06-13.
- [15] Raspberry pi-ventilador de refrigeración para cpu. <https://es.aliexpress.com/item/32813594463.html>. Accessed: 21-05-2024.
- [16] Kingston canvas select plus tarjeta microsd, sdcs2/32gb class 10. https://www.amazon.es/dp/B07ZG93S5J?psc=1&ref=ppx_yo2ov_dt_b_product_details. Accessed: 21-05-2024.
- [17] Precio de la luz en españa. <https://preciodelaluz.com/precio-por-meses>. Accessed: 2024-05-18.
- [18] Enlace de compra placas rpi 4 model b, 4 gb. <https://es.aliexpress.com/item/1005007384599192.html>.
- [19] Conmutador tp-link ls105g - switch ethernet 5 puertos (10/100/1000mbps). https://www.amazon.es/gp/product/B07RPVQY62/ref=ppx_yo_dt_b_asin_title_o03_s00?ie=UTF8&psc=1. Accessed: 21-05-2024.
- [20] Raspberry pi os. <https://www.raspberrypi.org/downloads/raspberry-pi-os/>. Accessed: 23-05-2024.
- [21] Raspberrypiimager. https://www.downloads.raspberrypi.org/raspios_arm64/images/. Accessed: 23-05-2024.
- [22] <https://www.digitalocean.com/community/tutorials/how-to-set-up-an-nfs-mount-on-ubuntu-20-04-es>. Accessed: 2024-08-11.
- [23] OpenMPI. OpenMPI. <https://www.open-mpi.org>. Accessed: 2024-05-28.
- [24] Cpufrequtils. <https://www.kernel.org/doc/Documentation/cpu-freq/governors.txt>.